

OpenBao — Operator Manual

What this technology does

OpenBao is the three-node KMS/secret-management service used by the Fabric evidence path. It stores the non-exportable Transit signing key, authenticates the Fabric adapter through a least-privilege AppRole, and maintains the cryptographic/audit boundary independently from Hyperledger Fabric.

OpenBao is **not** the Fabric ledger and is not the gateway-evidence verifier.

Current commissioned topology

```

ULC-01 10.104.0.2 -> OpenBao voter -> TLS API :8200 / Raft :8201
ULC-02 10.104.0.4 -> OpenBao voter -> TLS API :8200 / Raft :8201
ULC-03 10.104.0.8 -> OpenBao voter -> TLS API :8200 / Raft :8201

quorum = 2 of 3

ULC-01 adapter -> local HAProxy :18200 -> usable OpenBao backend
ULC-02 adapter -> local HAProxy :18200 -> usable OpenBao backend
TLS service name = openbao-kms.internal.lorawan.com

```

Current commissioned version: OpenBao 2.6.2 pinned by immutable image in the deployment runbook.

Runtime layout:

```

/etc/lorawan-cloud/openbao/openbao.hcl
/etc/lorawan-cloud/openbao/compose.yml
/etc/lorawan-pki/openbao/{ca.crt,server.crt,server.key}
/srv/openbao/data
container name: openbao

```

Security hard stops

Routine health work must **never** casually run:

```

bao operator init
bao operator unseal with shares pasted into a terminal
bao operator raft snapshot restore
Transit key delete/rotate/export operations
AppRole SecretID generation outside the documented rotation procedure
root-token extraction for ordinary troubleshooting
Raft data deletion/reset

```

Initialization occurs once per cluster. The protected bootstrap/recovery material must never be printed, committed to Git, copied into Markdown, or pasted into chat.

Step 1 — Check container state on all three nodes

Run on ULC-01, ULC-02, and ULC-03:

```
sudo docker compose -f /etc/lorawan-cloud/openbao/compose.yml ps
sudo docker inspect openbao --format 'status={{.State.Status}} restart_count={{.RestartCount}}'
```

PASS means: the OpenBao container is running on each intended member without a restart loop.

Step 2 — Check private listeners

On each node:

```
sudo ss -lntp | grep -E ':(8200|8201)\b'
```

Expected: the node's own private 10.104.0.x address, not a wildcard/public listener.

ULC-01 and ULC-02 should additionally have the HAProxy adapter-facing endpoint:

```
sudo ss -lntp | grep ':18200\b'
```

ULC-03 does not need an adapter-facing :18200 frontend.

Step 3 — Check direct node initialization/seal/HA state without a token

Use the current node IP:

```
NODE_IP="$(hostname -s | awk '{if($0=="ulc-01")print "10.104.0.2"; else if($0=="ulc-02")print "10.104.0.4";
else if($0=="ulc-03")print "10.104.0.8"}')"
```

```
test -n "$NODE_IP" || { echo 'UNKNOWN_HOST'; exit 1; }
```

```
sudo docker exec \
-e BAO_ADDR="https://${NODE_IP}:8200" \
-e BAO_CACERT=/openbao/tls/ca.crt \
openbao bao status -format=json
```

Interpret the JSON, not only the process exit code.

Normal commissioned result:

```
initialized = true
sealed = false
storage_type = raft
ha_enabled = true
```

A sealed OpenBao server can return a non-zero CLI status even though the process is running. That means the KMS is not usable; it is not permission to reinitialize it.

Step 4 — Check direct TLS health with a bounded timeout

```
NODE_IP="$(hostname -s | awk '{if($0=="ulc-01")print "10.104.0.2"; else if($0=="ulc-02")print "10.104.0.4";
else if($0=="ulc-03")print "10.104.0.8"}')"
```

```
curl --connect-timeout 3 --max-time 5 \
--cacert /etc/lorawan-pki/openbao/ca.crt \
--resolve "openbao-kms.internal.lorawan.com:8200:${NODE_IP}" \
-sS -o /tmp/openbao-health.json -w 'HTTP=%{http_code}\n' \
'https://openbao-kms.internal.lorawan.com:8200/v1/sys/health?standbyok=true'
```

```
cat /tmp/openbao-health.json
rm -f /tmp/openbao-health.json
```

PASS means: the initialized/unsealed node is classified usable under `standbyok=true`, with TLS hostname verification intact.

Do not use `-k`, `tls-skip-verify`, or an IP-name mismatch workaround.

Step 5 — Check the stable KMS endpoint from adapter hosts

On ULC-01:

```
curl --connect-timeout 3 --max-time 5 \
  --cacert /etc/haproxy/openbao-ca.crt \
  --resolve 'openbao-kms.internal.lorawan.com:18200:10.104.0.2' \
  -sS -o /dev/null -w 'HTTP=%{http_code}\n' \
  'https://openbao-kms.internal.lorawan.com:18200/v1/sys/health?standbyok=true'
```

On ULC-02, use `10.104.0.4` in `--resolve`.

PASS means: HTTP 200 through the node-local HAProxy KMS frontend with hostname verification.

If direct OpenBao nodes are healthy but `:18200` fails, troubleshoot HAProxy KMS routing before changing OpenBao.

Step 6 — Check recent bounded logs

```
sudo docker compose -f /etc/lorawan-cloud/openbao/compose.yml \
  logs --since=15m --tail=150 openbao
```

Look for seal, Raft, storage, TLS, audit, or request-forwarding errors. Do not dump secret-bearing request payloads.

Step 7 — Verify 3-voter Raft membership only with approved administrative authority

Raft membership is an authenticated administrative read. Use the protected operator procedure from the OpenBao runbook; do not extract the root token into ordinary shell history.

Required result:

```
exactly three intended voters
one current leader
ULC-01, ULC-02, ULC-03 all represented
```

The leader may move. Do not fail it back to match old documentation.

Step 8 — Verify the Transit/AppRole security contract

The commissioned contract is:

```
Transit key: lorawan-evidence
key type: ecdsa-p256
exportable: false
plaintext backup: false

policy: fabric-evidence-signer
allowed paths only:
  transit/sign/lorawan-evidence/sha2-256 -> update
  transit/verify/lorawan-evidence/sha2-256 -> update

AppRole: fabric-adapter
bind_secret_id: true
SecretID TTL: 24 h
```

Do not regenerate a SecretID merely because its current value is not visible. Inspect the adapter auth failure and follow the explicit credential-rotation procedure.

Important operational fact: issue/rotate the Fabric adapter SecretID against the **current active OpenBao member** using the reviewed helper. The current production history already proved that issuing against a standby can fail persistence against read-only storage.

Step 9 — Verify audit boundary before trusting signing evidence

Use the OpenBao runbook's audit inspection commands and confirm the commissioned audit device remains enabled/healthy and its storage path is protected.

Do not disable/re-enable an audit device to clear an error casually; doing so changes salt/HMAC continuity and can destroy incident-response evidence.

Safe configuration change procedure

1. prove all three members are initialized/unsealed and the stable endpoint works;
2. identify the current Raft leader with the approved admin read path;
3. back up the exact configuration being changed;
4. validate OpenBao HCL with the pinned image before restart;
5. validate Compose model;
6. change one node/candidate at a time;
7. preserve quorum throughout;
8. verify direct node health and stable :18200 health;
9. verify Raft membership;
10. verify one harmless scoped cryptographic operation only when the changed boundary requires it.

Do not restart all three members for a normal config change.

Backup/recovery rule

Before destructive KMS testing or recovery, use the dedicated runbook to create/verify a Raft snapshot and confirm protected recovery/unseal material exists off-node.

OpenBao Raft replication is HA, not a substitute for the protected recovery material or a tested recovery path.

Never perform an in-place snapshot restore merely because one node is unhealthy.

Troubleshooting map

Symptom	First check
container down	Compose/config/storage/TLS/runtime
process up, node sealed	seal/recovery procedure; not init
direct node healthy, :18200 fails	HAProxy KMS frontend/backend health
only adapter auth fails	AppRole RoleID/SecretID TTL/policy/login path
login works, sign denied	signer policy/key path/capabilities
signing works, Fabric fails	Fabric adapter/Fabric network, not OpenBao
inconsistent Raft peer view	authenticated Raft/member/network investigation

Completion checklist

- three OpenBao containers running;
- all three initialized and unsealed;
- private 8200/8201 listener exposure only;
- exact three-voter Raft cluster with one leader;
- ULC-01/02 stable :18200 KMS endpoints return TLS-verified health;
- lorawan-evidence key remains non-exportable ECDSA P-256;
- signer policy remains sign/verify-only;
- adapter AppRole remains least privilege;
- audit boundary is healthy;
- protected recovery/snapshot procedure is known before destructive work.

Detailed runbooks

- [../server/cloud-production/20a-openbao-three-node-ha-deployment.md](#)
- [../server/cloud-production/20-openbao-and-fabric-adapter.md](#)

Supporting runbook: 20a-openbao-three-node-ha-deployment.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

20A. OpenBao Three-Node HA Deployment Runbook

Status: COMPLETE / PASS FOR THE OPENBAO NORMAL PATH. Recorded execution below proves the three-node Raft cluster, private TLS, stable HAProxy KMS endpoints, Transit key/policy/AppRole boundary, and normal sign/verify acceptance. Keep all three members running. OpenBao member-loss, quorum-loss, restart/unseal recovery, Raft restore, adapter-loss, and Fabric-outage tests remain Phase 15 work; Fabric adapter deployment is a separate blocked Phase 20 boundary.

20A.1 Goal

Build one OpenBao Integrated Storage/Raft cluster across the existing three cloud hosts:

```
ulc-01 / 10.104.0.2 -> OpenBao-1 voter
ulc-02 / 10.104.0.4 -> OpenBao-2 voter
ulc-03 / 10.104.0.8 -> OpenBao-3 voter

Raft quorum = 2 of 3

adapter-1 on ulc-01 -> openbao-kms.internal.lorawan.com:18200 -> local HAProxy
adapter-2 on ulc-02 -> openbao-kms.internal.lorawan.com:18200 -> local HAProxy

HAProxy ulc-01/02 -> healthy initialized + unsealed OpenBao API backend :8200
OpenBao inter-node Raft/cluster traffic -> private :8201
```

OpenBao uses its own local persistent Raft storage on every host. Do not place the three voters on one shared writable volume and do not use PostgreSQL as the OpenBao storage backend for this POC.

20A.2 Fixed deployment values

The internal KMS service name is fixed independently from the still-unresolved public Internet domain:

```
KMS service name: openbao-kms.internal.lorawan.com
API:              private TLS TCP/8200
Raft cluster:     private TCP/8201
HAProxy frontend: private TCP/18200 on ulc-01 and ulc-02 only
```

OpenBao image pin prepared on 2026-08-27:

```
release tag:      2.6.2
image:            docker.io/openbao/openbao:2.6.2
OCI index digest: sha256:11fd73a2102cda9c55d5d881a8c3210303146a7ec1e8ac76f526e175c6d24641
```

```
linux/amd64:      sha256:e29524ba7c3f20d01f562c481e3eccbad6c91df45a2f2531433da4951e408cff
Compose pin:
docker.io/openbao/openbao@sha256:11fd73a2102cda9c55d5d881a8c3210303146a7ec1e8ac76f526e175c6d24641
```

Why this pin: 2.6.2 is the current patched 2.6 release at preparation time and fixes the critical internal-operation token-creation advisory affecting earlier releases. Do not replace this with latest, 2, or 2.6 during deployment. If execution occurs much later, re-check the current OpenBao security advisories first and record any deliberate pin change.

20A.3 What can be done now

This OpenBao subphase does **not** require:

```
physical gateway
SIM7600
Node-RED
Grafana
telemetry.fabric_outbox
Fabric adapter image
external Fabric handoff
```

It does require the already-existing cloud foundation:

```
[ ] ulc-01 / ulc-02 / ulc-03 reachable through the approved administration path
[ ] Docker + Compose already available
[ ] private VPC addresses 10.104.0.2 / .4 / .8 present
[ ] existing HA services remain healthy
[ ] TCP 8200 and 8201 unused on all three hosts
[ ] TCP 18200 unused on ulc-01 and ulc-02
[ ] enough free disk exists under /srv
```

Do not alter Patroni, etcd, PgBouncer, Valkey/Sentinel, Mosquitto, ChirpStack, or their already-commissioned listeners to make OpenBao fit.

20A.4 Read-only preflight — run on each node before mutation

Run this block separately on ulc-01, ulc-02, and ulc-03:

```
sudo -v && (
set -euo pipefail

NODE="$(hostname -s)"
case "$NODE" in
  ulc-01) NODE_IP='10.104.0.2'; NEED_18200=1 ;;
  ulc-02) NODE_IP='10.104.0.4'; NEED_18200=1 ;;
  ulc-03) NODE_IP='10.104.0.8'; NEED_18200=0 ;;
  *) echo "FAIL: unexpected host $NODE"; exit 1 ;;
esac

printf 'node=%s\nnode_ip=%s\n' "$NODE" "$NODE_IP"

ip -4 addr show | grep -F "$NODE_IP/" >/dev/null
printf 'PRIVATE_IP=PASS\n'

sudo docker version --format 'docker_server={{.Server.Version}}'
sudo docker compose version

for p in 8200 8201; do
  HITS="$(sudo ss -H -lnpt | awk -v port=":$p" '$4 ~ (port "$") {print}')"
  if [ -n "$HITS" ]; then
    echo "FAIL: TCP/$p already in use"
```

```

        printf '%s\n' "$HITS"
        exit 1
    fi
done

if [ "$NEED_18200" = 1 ]; then
    HITS="$(sudo ss -H -lntp | awk '$4 ~ /:18200$/ {print}')"
    if [ -n "$HITS" ]; then
        echo 'FAIL: TCP/18200 already in use'
        printf '%s\n' "$HITS"
        exit 1
    fi
fi

printf 'PORTS=PASS\n'

df -h / /srv 2>/dev/null || df -h /
free -h

for u in haproxy docker; do
    systemctl is-active "$u" 2>/dev/null || true
done

printf 'OPENBAO_PREFLIGHT=PASS\n'
)
RC=$?
echo "OPENBAO_PREFLIGHT_EXIT=$RC"
echo 'LOGIN_SHELL_SURVIVED=YES'

```

Stop on any listener collision. A port already in use must be identified before writing OpenBao configuration.

Recorded ulc-01 preflight - 2026-08-27

PASS. ulc-01 reported the expected private address 10.104.0.2; Docker Server 29.7.2 and Docker Compose v5.5.0 were available; TCP 8200, 8201, and 18200 were free; HAProxy and Docker were active; / had about 40G available; and Linux reported about 1.3 GiB available memory. The gate ended with OPENBAO_PREFLIGHT=PASS, OPENBAO_PREFLIGHT_EXIT=0, and the login shell survived. No OpenBao, HAProxy, firewall, or existing HA-service mutation occurred. Continue the same read-only gate on ulc-02 before any Phase 20A configuration mutation.

Recorded ulc-02 preflight - 2026-08-27

ulc-02 passed the Phase 20A read-only OpenBao preflight. Host identity resolved to ulc-02 / 10.104.0.4; Docker server 29.7.2 and Docker Compose v5.5.0 responded; TCP 8200, 8201, and 18200 were unused; HAProxy and Docker were active; root filesystem usage was 17% with about 40 GiB available; memory was about 1.9 GiB total with about 1.3 GiB available and no swap. Final gate: OPENBAO_PREFLIGHT=PASS, OPENBAO_PREFLIGHT_EXIT=0, LOGIN_SHELL_SURVIVED=YES. No service or configuration mutation occurred.

Recorded ulc-03 preflight - 2026-08-27

Observed from operator output:

```

node=ulc-03
node_ip=10.104.0.8
PRIVATE_IP=PASS
docker_server=29.7.2
Docker Compose version v5.5.0
DOCKER=PASS
PORTS=PASS
root filesystem: 48G total / 13G used / 36G available / 27%
memory: 1.9 GiB total / about 1.3 GiB available
swap: 0
haproxy=active
docker=active
OPENBAO_PREFLIGHT=PASS

```



```
OPENBAO_PREFLIGHT_EXIT=0
LOGIN_SHELL_SURVIVED=YES
```

Result: ulc-03 OPENBAO_PREFLIGHT=PASS. Together with the recorded ulc-01 and ulc-02 passes, the three-node read-only pre-mutation gate is **3/3 PASS**. The next allowed mutation is dedicated OpenBao PKI issuance on ulc-03.

20A.5 Dedicated OpenBao PKI

Use a dedicated OpenBao internal CA. Do not widen access to the PostgreSQL, MQTT, or Valkey CA private keys merely to save one certificate-generation step.

Create the CA and all three node identities on the protected administration/issuance host. For this POC the existing root-controlled issuance pattern on ulc-03 may be reused, but the OpenBao CA must have its own directory and key:

```
/root/lorawan-openbao-ca/
ca.key
ca.crt
issuance-<UTC>/
  ulc-01/{server.key,server.csr,server.crt}
  ulc-02/{server.key,server.csr,server.crt}
  ulc-03/{server.key,server.csr,server.crt}
```

Every server certificate must contain exactly the shared KMS name plus that physical node identity:

```
ulc-01: DNS:openbao-kms.internal.lorawan.com, DNS:ulc-01, IP:10.104.0.2
ulc-02: DNS:openbao-kms.internal.lorawan.com, DNS:ulc-02, IP:10.104.0.4
ulc-03: DNS:openbao-kms.internal.lorawan.com, DNS:ulc-03, IP:10.104.0.8
```

Required certificate properties:

```
serverAuth EKU
RSA-3072 or stronger
unique private key per node
unique random certificate serial per node
shared dedicated OpenBao CA
```

On ulc-03, create the dedicated CA and three node bundles without printing private-key contents:

```
sudo -v && sudo bash -c '
set -euo pipefail
umask 077

CA_ROOT=/root/lorawan-openbao-ca
ISSUE_DIR="$CA_ROOT/issuance-$(date -u +%Y%m%dT%H%M%SZ)"
mkdir -p "$ISSUE_DIR"
chmod 0700 "$CA_ROOT" "$ISSUE_DIR"

if { [ -e "$CA_ROOT/ca.key" ] && [ ! -s "$CA_ROOT/ca.crt" ]; } || { [ ! -s "$CA_ROOT/ca.key" ] && [ -e "$CA_ROOT/ca.crt" ]; }; then
  echo 'FAIL: partial OpenBao CA state exists; inspect before continuing'
  exit 1
fi

if [ ! -s "$CA_ROOT/ca.key" ] && [ ! -s "$CA_ROOT/ca.crt" ]; then
  openssl genrsa -out "$CA_ROOT/ca.key" 4096
  chmod 0600 "$CA_ROOT/ca.key"
  openssl req -x509 -new -sha256 \
    -key "$CA_ROOT/ca.key" \
    -days 3650 \
```

```

    -subj "/CN=LoRaWAN OpenBao Internal CA" \
    -out "$CA_ROOT/ca.crt"
    chmod 0644 "$CA_ROOT/ca.crt"
fi

for SPEC in \
    "ulc-01|10.104.0.2" \
    "ulc-02|10.104.0.4" \
    "ulc-03|10.104.0.8"
do
    NODE="${SPEC%|*}"
    IP="${SPEC##|}"
    D="$ISSUE_DIR/$NODE"
    mkdir -p "$D"

    openssl genrsa -out "$D/server.key" 3072
    chmod 0600 "$D/server.key"

    openssl req -new -sha256 \
        -key "$D/server.key" \
        -subj "/CN=openbao-kms.internal.lorawan.com" \
        -out "$D/server.csr"

    cat >"$D/ext.cnf" <<EOF
[v3_req]
basicConstraints = critical,CA:FALSE
keyUsage = critical,digitalSignature,keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = openbao-kms.internal.lorawan.com
DNS.2 = $NODE
IP.1 = $IP
EOF

    SERIAL="$(openssl rand -hex 16)"
    openssl x509 -req -sha256 \
        -in "$D/server.csr" \
        -CA "$CA_ROOT/ca.crt" \
        -CAkey "$CA_ROOT/ca.key" \
        -set_serial "0x$SERIAL" \
        -days 825 \
        -extfile "$D/ext.cnf" \
        -extensions v3_req \
        -out "$D/server.crt"

    cp "$CA_ROOT/ca.crt" "$D/ca.crt"
    chmod 0644 "$D/ca.crt" "$D/server.crt"

    openssl verify -CAfile "$D/ca.crt" "$D/server.crt"
    openssl x509 -in "$D/server.crt" -noout -checkhost openbao-kms.internal.lorawan.com
    openssl x509 -in "$D/server.crt" -noout -checkhost "$NODE"
    openssl x509 -in "$D/server.crt" -noout -checkip "$IP"

    CERT_HASH="$(openssl x509 -in "$D/server.crt" -pubkey -noout | openssl pkey -pubin -outform DER | sha256sum
| awk "{print \$1}")"
    KEY_HASH="$(openssl pkey -in "$D/server.key" -pubout -outform DER | sha256sum | awk "{print \$1}")"
    test "$CERT_HASH" = "$KEY_HASH"
    echo "$NODE certificate/key=PASS"
done

printf "OPENBAO_PKI_ISSUANCE=PASS\nISSUE_DIR=%s\n" "$ISSUE_DIR"

```

The generated ISSUE_DIR is the protected transfer source. Transfer only each node's ca.crt, server.crt, and server.key to that node. Do not transfer ca.key, CSRs, or another node's private key.

Before transfer, require for every node:

```
openssl verify -CAfile ca.crt server.crt = OK
openssl x509 -in server.crt -noout -checkhost openbao-kms.internal.lorawan.com = hostname match
openssl x509 -in server.crt -noout -checkhost <NODE> = hostname match
openssl x509 -in server.crt -noout -checkip <NODE_IP> = IP match
certificate public-key hash == private-key public-key hash
```

Do not place the OpenBao CA private key on u1c-01 or u1c-02, and do not commit any key, recovery share, token, AppRole SecretID, or certificate bundle containing a private key to Git.

Recorded Phase 20A.5 PKI issuance - 2026-08-27

PASS. Dedicated OpenBao PKI was created on u1c-03 under /root/lorawan-openbao-ca. The CA certificate SHA-256 is 18a8d9960b5a0bc0476e64628bdac0e00069aeae6b6ec7f0c95324fda119af6d and its SHA-256 fingerprint is 3D:85:BC:EB:B7:E3:FC:CB:41:E0:8F:48:86:28:1F:31:70:A8:F1:0C:2E:1C:E2:E9:26:AF:38:62:15:83:2B:17. The CA certificate/private-key public-key match passed.

Issuance directory: /root/lorawan-openbao-ca/issuance-20260827T050939Z.

- u1c-01 / 10.104.0.2: certificate fingerprint
55:13:21:08:98:20:6E:E1:BA:80:35:F6:4A:63:B5:51:46:4D:03:2C:0E:B6:D7:E3:0A:FF:A1:C9:BE:3E:B9:08;
serial FE8DE8801ED415D110C882ED5952120A.
- u1c-02 / 10.104.0.4: certificate fingerprint
27:3D:0D:FD:0A:E2:0D:7B:6F:DC:97:68:65:94:26:94:7C:FC:C5:FD:D3:4D:0D:0E:79:CB:67:8D:F0:64:7F:FD;
serial CDD04891F6A75AB12E9C9298C534C27E.
- u1c-03 / 10.104.0.8: certificate fingerprint
AD:FB:B7:90:E6:10:13:C7:56:E1:42:C0:98:2E:94:D7:63:CE:CB:8D:11:0F:25:1A:EB:B6:77:90:7C:FD:F2:1B;
serial FAC29163B53BBF297C90A4B567F77AD8.

All three certificates passed CA-chain verification, serverAuth purpose, shared hostname openbao-kms.internal.lorawan.com, node-hostname verification, node-IP verification, and certificate/private-key public-key equality. Unique certificate, private-key, and serial counts were all 3. OPENBAO_PKI_EXIT=0. The OpenBao CA private key remains only on u1c-03; do not copy it to any runtime node bundle.

20A.6 Pull and verify the immutable image on all three nodes

Run this first on u1c-03 as the image canary. After its digest/version checks pass, repeat the same immutable pull on u1c-01 and u1c-02 before creating runtime state:

```
set -euo pipefail

OPENBAO_IMAGE='docker.io/openbao/openbao@sha256:11fd73a2102cda9c55d5d881a8c3210303146a7ec1e8ac76f526e175c6d24641'
EXPECTED_INDEX='sha256:11fd73a2102cda9c55d5d881a8c3210303146a7ec1e8ac76f526e175c6d24641'
EXPECTED_AMD64='sha256:e29524ba7c3f20d01f562c481e3eccbad6c91df45a2f2531433da4951e408cff'

sudo docker pull "$OPENBAO_IMAGE"
sudo docker image inspect "$OPENBAO_IMAGE" --format 'image_id={{.Id}} repodigests={{json .RepoDigests}} user={{json .Config.User}}'

sudo docker buildx imagetools inspect 'docker.io/openbao/openbao:2.6.2' |
grep -F "$EXPECTED_INDEX"
sudo docker buildx imagetools inspect 'docker.io/openbao/openbao:2.6.2' |
grep -F "$EXPECTED_AMD64"

OPENBAO_UID="$(sudo docker run --rm --entrypoint sh "$OPENBAO_IMAGE" -c 'id -u openbao')"
OPENBAO_GID="$(sudo docker run --rm --entrypoint sh "$OPENBAO_IMAGE" -c 'id -g openbao')"

case "$OPENBAO_UID" in ''|*[!0-9]*) echo 'FAIL: invalid OpenBao UID'; exit 1 ;; esac
case "$OPENBAO_GID" in ''|*[!0-9]*) echo 'FAIL: invalid OpenBao GID'; exit 1 ;; esac
```

```
printf 'openbao_uid=%s\nopenbao_gid=%s\n' "$OPENBAO_UID" "$OPENBAO_GID"
printf 'OPENBAO_IMAGE_GATE=PASS\n'
```

The UID/GID is discovered from the exact pinned image instead of being guessed. Use the discovered numeric identity for host bind-mount ownership.

For OpenBao 2.6.x the container runs as the non-root openbao user by default. Because the observed container GID is 1000, do **not** make the server private key group-readable on the host: Ubuntu login accounts commonly also use a GID in that range. Install `server.key` as 0400 owned by the discovered OpenBao UID/GID, while the public CA/certificate remain root-owned and world-readable. This preserves non-root container execution without giving a host login group direct private-key read access.

Recorded Phase 20A.6 ulc-03 image canary - 2026-08-27

PASS. ulc-03 pulled the immutable OpenBao image

`docker.io/openbao/openbao@sha256:11fd73a2102cda9c55d5d881a8c3210303146a7ec1e8ac76f526e175c6d24641`.

Local RepoDigest matched the expected OCI index, platform was `linux/amd64`, and bao version reported OpenBao v2.6.2 commit `dd9c19c37a878cf4a81b18efb8d6f0599c7da923` committed 2026-08-18T15:48:19Z. TCP 8200/8201 remained free and no OpenBao service/container runtime state was created. `OPENBAO_IMAGE_CANARY_EXIT=0`.

For the remaining two nodes, the existing root-controlled deployment key on ulc-03, `/root/.ssh/cloud-deployment-phase8`, may be used to drive sequential SSH image pulls through `opsadmin@10.104.0.2/.4` with `IdentitiesOnly=yes` and `StrictHostKeyChecking=yes`. Keep remote `sudo` interactive if the hardened account requires its password; do not weaken `sudo` or SSH controls merely to make the rollout non-interactive. Verify one node fully before proceeding to the other.

Recorded Phase 20A.6 three-node image rollout - 2026-08-27

PASS. The exact pinned OpenBao v2.6.2 image digest

`sha256:11fd73a2102cda9c55d5d881a8c3210303146a7ec1e8ac76f526e175c6d24641` is now present on ulc-01, ulc-02, and ulc-03. ulc-01 and ulc-02 independently verified `linux/amd64`, reported OpenBao runtime identity `UID=100 / GID=1000`, and proved no OpenBao container or 8200/8201/18200 listener was created by the pull. The earlier ulc-03 canary verified the same digest/platform/version and no runtime state. `OPENBAO_IMAGE_3_OF_3=PASS`; `OPENBAO_SERVICE_STARTED=NO`.

20A.7 Install node TLS material and runtime directories

On each node, after its own `ca.crt`, `server.crt`, and `server.key` have arrived through the approved protected transfer path:

```
sudo -v && (
set -euo pipefail

OPENBAO_IMAGE='docker.io/openbao/openbao@sha256:11fd73a2102cda9c55d5d881a8c3210303146a7ec1e8ac76f526e175c6d24641'
OPENBAO_UID="$(sudo docker run --rm --entrypoint sh "$OPENBAO_IMAGE" -c 'id -u openbao')"
OPENBAO_GID="$(sudo docker run --rm --entrypoint sh "$OPENBAO_IMAGE" -c 'id -g openbao')"

sudo install -d -m 0755 -o root -g root /etc/lorawan-pki/openbao
sudo install -d -m 0755 -o root -g root /etc/lorawan-cloud/openbao
sudo install -d -m 0700 -o "$OPENBAO_UID" -g "$OPENBAO_GID" /srv/openbao/data

# Install only the three approved files. The private key is owner-readable by
# the OpenBao container UID, not group-readable by host GID 1000.
sudo install -m 0444 -o root -g root ./ca.crt /etc/lorawan-pki/openbao/ca.crt
sudo install -m 0444 -o root -g root ./server.crt /etc/lorawan-pki/openbao/server.crt
sudo install -m 0400 -o "$OPENBAO_UID" -g "$OPENBAO_GID" ./server.key /etc/lorawan-pki/openbao/server.key

sudo test ! -e /etc/lorawan-pki/openbao/ca.key
```

```

sudo test "$(stat -c %a /etc/lorawan-pki/openbao/server.key)" = 400
sudo docker run --rm --user "$OPENBAO_UID:$OPENBAO_GID" \
  -v /etc/lorawan-pki/openbao:/openbao/tls:ro \
  --entrypoint sh "$OPENBAO_IMAGE" \
  -c 'test -r /openbao/tls/ca.crt && test -r /openbao/tls/server.crt && test -r /openbao/tls/server.key'

printf 'OPENBAO_RUNTIME_DIRS=PASS\n'
)

```

On ulc-01 and ulc-02, also install a **public CA-only** copy for HAProxy without granting HAProxy traversal of the OpenBao server-key directory:

```

sudo install -m 0644 -o root -g root \
  /etc/lorawan-pki/openbao/ca.crt \
  /etc/haproxy/openbao-ca.crt

```

Recorded Phase 20A.7 TLS/runtime staging - 2026-08-27

PASS. ulc-03 was staged locally first, followed sequentially by ulc-01 and ulc-02. Every node re-verified its dedicated OpenBao CA chain, shared service hostname, node hostname, node IP, and certificate/private-key match. The exact image runtime identity was Config.User=openbao, UID=100, GID=1000. Installed permissions were 0444 root:root for ca.crt and server.crt, 0400 100:1000 for server.key, and 0700 100:1000 for /srv/openbao/data. The non-root image user proved it could read all TLS files and write the Raft data directory. ulc-01/02 received only the public CA copy for future HAProxy health checks. Each node independently returned OPENBAO_NODE_STAGING=PASS, no OpenBao container/listener was created, and the enclosing command returned PHASE20A7_EXIT=0. The operator output did not include the optional final aggregate echo block, so acceptance is based on the three explicit per-node PASS gates plus exit 0, not on an unobserved aggregate line.

20A.8 Write node-specific OpenBao configuration

Run on each node. The script derives the correct private address and the two alternative join targets from the physical hostname:

```

sudo -v && (
set -euo pipefail

NODE="$(hostname -s)"
case "$NODE" in
  ulc-01)
    NODE_IP='10.104.0.2'
    PEER1='10.104.0.4'
    PEER2='10.104.0.8'
    ;;
  ulc-02)
    NODE_IP='10.104.0.4'
    PEER1='10.104.0.2'
    PEER2='10.104.0.8'
    ;;
  ulc-03)
    NODE_IP='10.104.0.8'
    PEER1='10.104.0.2'
    PEER2='10.104.0.4'
    ;;
  *) echo "FAIL: unexpected node $NODE"; exit 1 ;;
esac

OPENBAO_UID="$(sudo docker run --rm --entrypoint sh \
  'docker.io/openbao/openbao@sha256:11fd73a2102cda9c55d5d881a8c3210303146a7ec1e8ac76f526e175c6d24641' \
  -c 'id -u openbao'))"
OPENBAO_GID="$(sudo docker run --rm --entrypoint sh \
  'docker.io/openbao/openbao@sha256:11fd73a2102cda9c55d5d881a8c3210303146a7ec1e8ac76f526e175c6d24641' \
  -c 'id -g openbao'))"

```

```

TMP="$(mktemp)"
trap 'rm -f "$TMP"' EXIT

cat >"$TMP" <<EOF
ui = false
api_addr      = "https://${NODE_IP}:8200"
cluster_addr  = "https://${NODE_IP}:8201"

storage "raft" {
  path      = "/openbao/data"
  node_id   = "${NODE}"

  retry_join {
    leader_api_addr      = "https://${PEER1}:8200"
    leader_tls_servername = "openbao-kms.internal.lorawan.com"
    leader_ca_cert_file   = "/openbao/tls/ca.crt"
  }

  retry_join {
    leader_api_addr      = "https://${PEER2}:8200"
    leader_tls_servername = "openbao-kms.internal.lorawan.com"
    leader_ca_cert_file   = "/openbao/tls/ca.crt"
  }
}

listener "tcp" {
  address            = "${NODE_IP}:8200"
  cluster_address    = "${NODE_IP}:8201"
  tls_cert_file      = "/openbao/tls/server.crt"
  tls_key_file        = "/openbao/tls/server.key"
  tls_min_version    = "tls12"
}
EOF

sudo install -m 0400 -o "$OPENBAO_UID" -g "$OPENBAO_GID" \
  "$TMP" /etc/lorawan-cloud/openbao/openbao.hcl

printf 'OPENBAO_CONFIG_WRITTEN=PASS\n'
)

```

Why two `retry_join` targets: a joining/restarting node must not depend on one particular peer being active. With Shamir sealing, successful Raft join does **not** remove the need to unseal the joined node manually.

20A.9 Create the Docker Compose unit

Use the same Compose definition on all three nodes:

```

services:
  openbao:
    image: docker.io/openbao/openbao@sha256:11fd73a2102cda9c55d5d881a8c3210303146a7ec1e8ac76f526e175c6d24641
    container_name: openbao
    network_mode: host
    restart: unless-stopped
    command: ["server"]
    environment:
      SKIP_CHOWN: "1"
    volumes:
      - /etc/lorawan-cloud/openbao:/openbao/config:ro
      - /etc/lorawan-pki/openbao:/openbao/tls:ro
      - /srv/openbao/data:/openbao/data
    mem_limit: 512m
    pids_limit: 256
    stop_grace_period: 60s
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true
    logging:
      driver: json-file

```

```
options:
  max-size: "10m"
  max-file: "5"
```

Install it as `/etc/lorawan-cloud/openbao/compose.yml`, then validate both the OpenBao HCL and Compose model before any start. The exact pinned OpenBao CLI provides `operator validate-config`; use it against the bind-mounted file so listener TLS, addresses, and server configuration are parsed by the same release that will run the service:

```
sudo docker run --rm \
  -v /etc/lorawan-cloud/openbao:/openbao/config:ro \
  -v /etc/lorawan-pki/openbao:/openbao/tls:ro \
  -v /srv/openbao/data:/openbao/data \
  --entrypoint bao \
  docker.io/openbao/openbao@sha256:11fd73a2102cda9c55d5d881a8c3210303146a7ec1e8ac76f526e175c6d24641 \
  operator validate-config -config=/openbao/config/openbao.hcl

sudo docker compose -f /etc/lorawan-cloud/openbao/compose.yml config --quiet
```

Why host networking: the Raft/API addresses remain the already-defined VPC addresses and do not depend on Docker bridge/NAT addresses. The OpenBao listener itself still binds only the node's private VPC IP; there is no wildcard or public API listener.

Why container command is only `server`: the official OpenBao Docker entrypoint automatically adds `-config=/openbao/config` when the first argument is `server`. Adding a second explicit `-config=/openbao/config/openbao.hcl` causes the same file to be discovered twice and produces the observed duplicate-configuration warning. Keep only command: `["server"]`; the bind-mounted `/openbao/config/openbao.hcl` is loaded automatically. This correction must be applied before any later restart/start of a node, but do not restart the currently wedged `ulc-01` solely to normalize the command while recovery evidence is still being collected.

OpenBao removed `mlock` support in the 2.x line, so do not add obsolete `IPC_LOCK` capability merely because older Vault/OpenBao examples show it.

Recorded Phase 20A.8/9 configuration validation - 2026-08-27

PASS. `ulc-03`, then `ulc-01`, then `ulc-02` each passed TLS/runtime baseline verification, node-specific `openbao.hcl` creation, immutable-image config validation (`[ success ] Validate Config`), and Docker Compose model validation using

`docker.io/openbao/openbao@sha256:11fd73a2102cda9c55d5d881a8c3210303146a7ec1e8ac76f526e175c6d24641`. Each node proved no OpenBao container/runtime listener was started by configuration. Operator result: `PHASE20A89_EXIT=0; LOGIN_SHELL_SURVIVED=YES`.

The next allowed state change is to start **`ulc-01` only** and verify the expected uninitialized/sealed seed-node state before the one-time initialization. Do not start `ulc-02` or `ulc-03` yet.

Recorded Phase 20A.10 pre-initialization seed gate - 2026-08-27

PASS. `ulc-01` started successfully from the pinned OpenBao v2.6.2 image. The TLS API returned HTTP 501, `/v1/sys/init` returned `{"initialized":false}`, and `bao status -format=json` returned exit code 2 with `initialized=false`, `sealed=true`, `storage_type=raft`, and `ha_enabled=true`. The container was running with restart count 0; the API listener bound to `10.104.0.2:8200`. `ulc-02` and `ulc-03` remained stopped. `OPENBAO_SEED_PREINIT_GATE=PASS; OPENBAO_INITIALIZATION_EXECUTED=NO`.

The retry-join connection-refused messages to `10.104.0.4:8200` and `10.104.0.8:8200` are expected at this checkpoint because those two peers are intentionally still stopped. The log warning ignoring duplicate configuration found in directory: `/openbao/config/openbao.hcl` is non-fatal. It indicates the same HCL file was discovered more than once by the current container invocation; normalize the Compose invocation before a

later restart so the warning is removed, but do not restart the successfully running seed before initialization solely for this warning.

20A.10 Start and initialize OpenBao-1 only

Start **ulc-01 only**. Do not initialize in the same operator checkpoint. First prove the seed server is reachable over the dedicated CA and remains uninitialized/sealed while ulc-02 and ulc-03 are still stopped.

Required pre-initialization state:

```
ulc-01 container: running
ulc-01 API TLS: verified
ulc-01 Initialized: false
ulc-01 Sealed: true
ulc-02 OpenBao: stopped
ulc-03 OpenBao: stopped
```

The OpenBao CLI status command returns exit code 2 for a sealed server; do not treat that expected code as a failed start. Before initialization, `/v1/sys/health` should classify the node as uninitialized. Do not override the uninitialized/sealed health codes merely to make this pre-init state look green.

Operator shell-safety rule

Do **not** paste `set -euo pipefail` directly into the interactive `opsadmin` login shell. `set -e` applies to that shell itself; any later non-zero command can terminate the SSH session. For all remaining Phase 20A operator blocks, put strict mode inside a child script or subshell, run that child from the login shell, capture its exit code, then print `LOGIN_SHELL_SURVIVED=YES`.

If an SSH terminal closes during a sensitive step, reconnect and inspect current state before repeating the mutation. For initialization specifically, inspect both `/root/lorawan-openbao-bootstrap/init.json` and `/root/lorawan-openbao-bootstrap/.init.json.tmp` plus `/v1/sys/init`; never rerun `bao operator init` merely because the controlling terminal disappeared.

Operator correction - Phase 20A.10 stdin safety

The first ulc-03-driven seed-start attempt returned `PHASE20A10_PREINIT_EXIT=0` after only proving ulc-02 and ulc-03 stopped. It did **not** start ulc-01. Root cause: a non-interactive `ssh` command ran while the parent `sudo bash <<'OUTER'` still received its script from stdin; SSH consumed the remaining heredoc, so the parent shell reached a clean EOF. Treat that attempt as **INCOMPLETE / NO START**.

For later ulc-03-driven wrappers:

- do not place interactive SSH inside an outer heredoc-fed shell;
- use `ssh -n` (or stdin from `/dev/null`) for non-interactive checks;
- finish any heredoc that creates a remote script before invoking `ssh -tt`;
- require explicit ulc-01 TLS/API, `Initialized=false`, and `Sealed=true` evidence before initialization.

Initialize **exactly once** on ulc-01. The operator block must refuse to overwrite an existing bootstrap file, write JSON with `umask 077`, never print unseal shares or the root token, verify three Shamir shares with threshold two, and record only a SHA-256 integrity hash plus non-secret state. After initialization the seed remains sealed until the separate unseal checkpoint.

Acceptance after initialization:


```
bootstrap file exists and is mode 0600 root:root
3 unseal shares generated
unseal threshold = 2
root token present but never printed
/v1/sys/init initialized=true
bao status initialized=true, sealed=true
ulc-02 and ulc-03 still stopped
```

Do **not** initialize ulc-02 or ulc-03. Three independent initializations would create three unrelated clusters.

Operator correction - recovery probe timeout safety

The first initialization-recovery probe could stall at GET /v1/sys/init because its curl invocation had no connection or total timeout. A read-only initialization-state query should return promptly; a prolonged wait is not an acceptable gate. Interrupting that diagnostic with Ctrl+C is safe because no initialization mutation is being executed in that recovery check.

All later OpenBao recovery/health probes must use bounded waits, for example curl --connect-timeout 3 --max-time 5, and CLI/API checks that could block should be wrapped with timeout. A timeout means **state unknown**, not initialized or uninitialized. Never infer state from a hung request and never rerun bao operator init until both the protected bootstrap-file state and /v1/sys/init have been rechecked successfully.

If the nested SSH session closes before the initialization result is visible, do **not** rerun bao operator init blindly. First perform a read-only recovery check from ulc-03 that verifies both /root/lorawan-openbao-bootstrap/init.json existence on ulc-01 and the current /v1/sys/init / bao status state. A normal Connection to 10.104.0.2 closed. after the remote command only means the nested SSH session ended and control returned to ulc-03.

The initialization file contains the three Shamir unseal shares and initial root token. Move this bootstrap material through the approved protected administration path before normal operation. Do not paste it into chat, Git, shell history, Node-RED, Grafana, or Fabric configuration.

Recorded Phase 20A initialization recovery attempt 1 - 2026-08-27

INCOMPLETE / READ-ONLY. The first recovery diagnostic confirmed the ulc-01 OpenBao container was still running, then the /v1/sys/init curl probe stalled until the operator interrupted it with Ctrl+C. The nested SSH returned exit 255; cleanup completed; ulc-02 and ulc-03 remained stopped. No initialization or unseal action was executed by this recovery diagnostic. Treat initialization state as **UNKNOWN** until a bounded recovery check verifies bootstrap-file presence plus API/CLI state.

Recorded Phase 20A initialization recovery attempt 2 - 2026-08-27

UNKNOWN / DO NOT RETRY INIT. The bounded recovery check found /root/lorawan-openbao-bootstrap/init.json absent and /root/lorawan-openbao-bootstrap/.init.json.tmp present with mode 0600 root:root but size 0 and SHA-256 e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855. openbao remained running with restart count 0. Both bao status (6-second bound) and /v1/sys/init (4-second curl bound) timed out. The recovery decision was RECOVERY_STATE=UNKNOWN_REQUIRES_REVIEW; DO_NOT_RERUN_OPERATOR_INIT=YES. ulc-02 and ulc-03 were not started.

Interpretation: the prior bao operator init attempt opened the protected temporary output file but did not complete far enough to emit JSON. The next step is read-only process/log/Raft-state inspection on ulc-01. Do not delete the temporary file, restart OpenBao, rerun operator init, or modify /srv/openbao/data until that inspection is reviewed.

The read-only inspection must capture: host/container processes related to bao operator init or docker exec; docker top for the OpenBao container; current /srv/openbao/data file names, sizes, timestamps, and permissions

without modifying them; bounded TCP/TLS reachability; and recent OpenBao logs around the initialization attempt. Do not use `kill`, `docker restart`, `docker stop`, `rm`, `mv`, or any Raft recovery command in this inspection.

Recorded Phase 20A initialization recovery attempt 3 - 2026-08-27

PARTIAL RAFT BOOTSTRAP / CORE UNRESPONSIVE. `ulc-01` remained running with restart count 0; there was no lingering host-side `bao operator init`, `docker exec`, or `bao status` process. The container contained only the OpenBao server process. `/root/lorawan-openbao-bootstrap/init.json` was absent and `.init.json.tmp` remained mode `0600 root:root`, size 0. Integrated-storage files had nevertheless been created: `/srv/openbao/data/raft/raft.db` and `/srv/openbao/data/vault.db`, each 16,801,792 bytes with modification time around 2026-08-27T05:38:59Z. The TLS listener on `10.104.0.2:8200` remained present and an OpenSSL handshake verified successfully, but the bounded HTTP health request timed out and ended the diagnostic with `curl` exit 28. Treat this as a wedged/indeterminate partial bootstrap. Do not rerun `operator init`, restart OpenBao, delete the temporary bootstrap file, or alter Raft data until logs/resource evidence is reviewed.

Recorded Phase 20A initialization recovery attempt 4 - 2026-08-27

LOST BOOTSTRAP SECRETS / RESET REQUIRED BEFORE RETRY. Resource evidence ruled out host pressure: the OpenBao container was not OOM-killed, restart count remained 0, memory use was about 16 MiB / 512 MiB, host memory still showed about 1.2 GiB available, and no relevant kernel/storage errors were found. The initialization-time log proves the one-node Raft bootstrap crossed the initialization boundary: `ulc-01` created Raft with itself as the only voter, won leadership, acquired the HA initialization lock, and logged security barrier initialized: `shares=3 threshold=2`. It then entered post-unseal setup and mounted default `cubbyhole/`, `sys/`, and `identity/` paths, but the operator-side protected temporary output file remained zero bytes. No usable Shamir shares or root token were captured.

Because this is a brand-new OpenBao cluster with no production secrets, no joined peers, and no usable bootstrap material, the correct recovery is **not** to retry `operator init` against the existing Raft state. Preserve a root-only forensic archive of the partial Raft state and logs, then stop OpenBao, reset only `/srv/openbao/data` on `ulc-01`, remove only the zero-byte temporary bootstrap file, correct the seed configuration, and return to a clean `Initialized=false / Sealed=true` gate before a new one-time initialization. Do not touch PostgreSQL, Patroni, etcd, PgBouncer, Valkey, Mosquitto, ChirpStack, HAProxy, or the OpenBao PKI CA.

Seed configuration correction for the clean retry:

- `ulc-01` initially has **no `retry_join` stanzas**. The documented Raft flow is to initialize one node as leader first; remaining uninitialized nodes then use `retry_join` to locate that leader.
- `ulc-02` and `ulc-03` retain their `retry_join` configuration for the later join phase.
- Compose uses only `command: ["server"]`; the official container entrypoint already adds `-config=/openbao/config`, so the previous explicit second `-config` argument caused the duplicate-configuration warning.
- Do not initialize in the same checkpoint as the reset/restart. First prove the clean seed again.

Acceptance after controlled reset:

```
ulc-01 OpenBao running with restart_count=0
no duplicate-configuration warning on the new start
no retry_join attempts from ulc-01 on the new start
/v1/sys/init -> initialized=false
bao status -> initialized=false, sealed=true
bootstrap init.json absent
bootstrap .init.json.tmp absent
ulc-02 OpenBao stopped
ulc-03 OpenBao stopped
```

Recorded Phase 20A controlled seed recovery - 2026-08-27

PASS. The failed first initialization was preserved under `/root/lorawan-openbao-recovery/partial-bootstrap-20260827T061259Z`, including the zero-byte temp bootstrap file, pre-fix config/Compose, logs, and archived partial Raft state with SHA-256 verification. Only the fresh OpenBao seed state on ulc-01 was reset. The seed HCL now has no `retry_join`, Compose now uses only command: `["server"]`, and both corrected files validated with OpenBao v2.6.2 / Docker Compose. The clean restarted seed returned health HTTP 501, `initialized=false`, `sealed=true`, restart count 0, no duplicate-config warning, and no retry-join activity. ulc-02 and ulc-03 remained stopped. `PHASE20A_SEED_RECOVERY=PASS`; `PHASE20A_SEED_RECOVERY_EXIT=0`; `LOGIN_SHELL_SURVIVED=YES`.

The next allowed mutation is a second one-time initialization attempt on ulc-01 only. The secret-producing command must run entirely on ulc-01, write to a root-only temporary file, `fsync/validate` it, then atomically rename it to `init.json`. The wrapper must not terminate the ulc-03 login shell on failure and must never print secrets.

Second initialization attempt - execution safety

Run the actual `bao operator init` inside a root-owned transient systemd service on ulc-01. The service must redirect stdout directly to `/root/lorawan-openbao-bootstrap/.init.json.tmp`, redirect stderr to a separate root-only error file, validate the JSON without printing it, `fsync` the file, then atomically rename it to `init.json`. The ulc-03 SSH wrapper only launches and polls the transient service; losing that SSH session must not terminate the secret-producing `init` process. Do not put a timeout around `bao operator init` itself because killing the client after the server has crossed the initialization boundary could reproduce the lost-bootstrap-secret failure. Timeouts remain required only for read-only API/status probes.

Before launching the service, re-prove `initialized=false`, `sealed=true`, no bootstrap files, and ulc-02/ulc-03 stopped. After a successful service exit, require a mode-0600 root-owned `init.json` with three unique base64 unseal shares and a non-empty root token, but print only counts/presence and a SHA-256 integrity hash. Then require OpenBao `initialized=true`, `sealed=true`, `shares=3`, `threshold=2`.

Recorded Phase 20A second protected initialization - 2026-08-27

PASS. After the controlled seed reset, the second one-time initialization ran as transient systemd unit `lorawan-openbao-init-20260827T062342Z.service` on ulc-01. The unit completed with `Result=success` and `ExecMainStatus=0`. The protected bootstrap file was atomically finalized as `/root/lorawan-openbao-bootstrap/init.json`, mode 0600, containing three unique Shamir shares with threshold two and a present initial root token; none of those secrets were printed. Recorded file SHA-256: `66045a7bd3cd715c198fe2ad1c536bfa535aa96bcab47ddd8abf8b7ea5ad9831`. OpenBao reported `initialized=true`, `sealed=true`, `storage_type=raft`, `ha_enabled=true`; `/v1/sys/health` returned 503, which is the expected sealed state. ulc-02 and ulc-03 remained stopped. `PHASE20A_SECOND_INITIALIZATION=PASS`; `PHASE20A_SECOND_INIT_EXIT=0`; `LOGIN_SHELL_SURVIVED=YES`.

The next allowed mutation is Phase 20A.11: unseal **ulc-01 only** with two distinct shares sourced directly from the root-only `init.json`. Do not place shares in command-line arguments, shell history, terminal output, chat, or logs. After two shares, require `Initialized=true`, `Sealed=false`, and `HA Enabled=true` before starting any peer.

20A.11 Unseal OpenBao-1

Use the two required Shamir shares directly from the root-only `/root/lorawan-openbao-bootstrap/init.json` on ulc-01. Do not copy shares into the terminal, command arguments, shell history, chat, or logs. The preferred commissioning method is a root-only local script that reads two distinct shares in memory and submits each share to `PUT /v1/sys/unseal` over the node certificate/CA. Print only non-secret fields (`sealed`, `progress`, `t`, `n`).

Required sequence:

1. re-verify ulc-01 is initialized and sealed and ulc-02/03 remain stopped;

2. submit share 1 and require sealed=true, progress=1, t=2, n=3;
3. submit a different share 2 and require sealed=false;
4. verify bao status reports initialized=true, sealed=false, storage_type=raft, ha_enabled=true;
5. require HTTPS health 200 and private listeners on 10.104.0.2:8200 and 10.104.0.2:8201;
6. do not start ulc-02 or ulc-03 in the same checkpoint.

Why local API submission: unseal is stateful and each Shamir share is accepted independently. Reading shares from the protected file avoids exposing them through command-line arguments or operator copy/paste.

Recorded Phase 20A.11 ulc-01 unseal - 2026-08-27

PASS. The protected root-only bootstrap file on ulc-01 was verified as mode 0600 root:root with three unique Shamir shares. Two distinct shares were submitted locally without printing them. After the first share OpenBao remained sealed with progress=1; after the second share the threshold was reached and sealed=false. bao status returned exit 0 with initialized=true, sealed=false, storage_type=raft, and ha_enabled=true. The dedicated TLS health endpoint returned HTTP 200, private listeners were present on 10.104.0.2:8200 and 10.104.0.2:8201, the container remained stable with restart count 0, and the bootstrap file remained protected. ulc-02 and ulc-03 remained stopped. PHASE20A11_UNSEAL=PASS; PHASE20A11_UNSEAL_EXIT=0; LOGIN_SHELL_SURVIVED=YES.

The next allowed state change is to start **ulc-02 only**. Before that start, normalize its Compose command to command: ["server"] so it does not inherit the duplicate-config invocation. Keep ulc-02 retry_join entries so it can discover the active ulc-01 leader. Do not copy init.json or the root token to ulc-02; submit exactly two Shamir shares from the protected ulc-01 bootstrap file directly to ulc-02's /v1/sys/unseal API after join.

20A.12 Start/join/unseal OpenBao-2, then OpenBao-3

Bring peers into the Raft cluster **one node at a time**. Complete and verify ulc-02 before any ulc-03 start.

20A.12A ulc-02 normalize and join while sealed

Before starting ulc-02:

- prove ulc-03 is still stopped;
- prove ulc-01 is initialized, unsealed, healthy, and still owns the protected bootstrap file;
- prove ulc-02 has no existing Raft state and ports 8200/8201 are free;
- preserve ulc-02's two retry_join stanzas, including the active ulc-01 target;
- normalize ulc-02 Compose to command: ["server"] before its first start, then validate the exact HCL and Compose model.

Start ulc-02 only. Its retry_join must attach it to the existing ulc-01 cluster. Before submitting any Shamir shares, require ulc-02 to report initialized=true, sealed=true, and require the authenticated Raft configuration read from ulc-01 to show exactly two voters: ulc-01 and ulc-02. A sealed joined peer is expected to return sealed health rather than normal HTTP 200.

20A.12B unseal ulc-02 without distributing bootstrap material

Do not copy /root/lorawan-openbao-bootstrap/init.json to ulc-02. Keep all bootstrap material only on ulc-01. A root-only helper on ulc-01 reads two distinct Shamir shares from that file and submits them directly over the private TLS API to https://10.104.0.4:8200/v1/sys/unseal. Never print shares or place them in command arguments.

After share one, require sealed=true and progress=1. After share two, require sealed=false. Then verify ulc-02 returns normal usable health with standbyok=true, listens privately on 8200/8201, remains restart-count 0, and the Raft configuration still contains exactly two voters with one leader. Re-prove ulc-03 is stopped.

Required ulc-02 gate:

```
ulc-01 initialized=true sealed=false healthy
ulc-02 initialized=true sealed=false healthy with standbyok=true
Raft voters: ulc-01, ulc-02 exactly
ulc-03 OpenBao: stopped
bootstrap init.json remains only on ulc-01
```

20A.12C ulc-03

Only after the ulc-02 gate passes, repeat the same pattern for ulc-03: normalize Compose, start it, verify it joins while sealed, submit two shares from ulc-01 without copying bootstrap material, then verify exactly three voters ulc-01, ulc-02, ulc-03 and direct usable health on all three nodes.

Do not perform destructive leader-kill, quorum-loss, or host-loss testing here. Those belong to the later failure-testing phase.

20A.13 Enable Transit and create the non-exportable evidence key

20A.13A Read-only preflight before any administrative mutation

Do not run the original all-in-one bootstrap block blindly. First prove the three-node cluster is still healthy and inspect whether transit/, approle/, lorawan-evidence, fabric-evidence-signer, or the fabric-adapter role already exists. Load the initial root token only inside a root-owned child script or protected variable, never print it, and unset it immediately after the read-only checks.

Proceed with Phase 20A.13 mutations sequentially: enable/verify transit/; create/verify the key; write/verify the signer policy; enable/verify AppRole; create/verify the role. Stop after any mismatch. Do not issue a SecretID in this phase.

Perform administrative configuration on ulc-01 once the 3/3 cluster is healthy. Load the initial root token from the protected bootstrap file into a shell variable without printing it, perform the bootstrap operations, then immediately unset it.

```
sudo -v && (
set -euo pipefail

ROOT_TOKEN="$(sudo python3 -c 'import json; print(json.load(open("/root/lorawan-openbao-bootstrap/init.json",
"r", encoding="utf-8"))["root_token"])'")
trap 'unset ROOT_TOKEN' EXIT

BAO_ADDR='https://10.104.0.2:8200'
CA='/openbao/tls/ca.crt'

sudo docker exec \
-e BAO_ADDR="$BAO_ADDR" -e BAO_CACERT="$CA" -e BAO_TOKEN="$ROOT_TOKEN" \
openbao bao secrets enable -path=transit transit

sudo docker exec \
-e BAO_ADDR="$BAO_ADDR" -e BAO_CACERT="$CA" -e BAO_TOKEN="$ROOT_TOKEN" \
openbao bao write -f transit/keys/lorawan-evidence \
type=ecdsa-p256 exportable=false allow_plaintext_backup=false

cat <<'HCL' | sudo docker exec -i \
-e BAO_ADDR="$BAO_ADDR" -e BAO_CACERT="$CA" -e BAO_TOKEN="$ROOT_TOKEN" \
openbao bao policy write fabric-evidence-signer -
path "transit/sign/lorawan-evidence/sha2-256" {
  capabilities = ["update"]
}
path "transit/verify/lorawan-evidence/sha2-256" {
  capabilities = ["update"]
}
```

HCL

```
sudo docker exec \  
-e BAO_ADDR="$BAO_ADDR" -e BAO_CACERT="$CA" -e BAO_TOKEN="$ROOT_TOKEN" \  
openbao bao auth enable approle  
  
sudo docker exec \  
-e BAO_ADDR="$BAO_ADDR" -e BAO_CACERT="$CA" -e BAO_TOKEN="$ROOT_TOKEN" \  
openbao bao write auth/approle/role/fabric-adapter \  
token_policies=fabric-evidence-signer \  
token_ttl=1h token_max_ttl=4h \  
secret_id_ttl=24h secret_id_num_uses=0  
  
sudo docker exec \  
-e BAO_ADDR="$BAO_ADDR" -e BAO_CACERT="$CA" -e BAO_TOKEN="$ROOT_TOKEN" \  
openbao bao read transit/keys/lorawan-evidence  
  
echo 'OPENBAO_TRANSIT_BOOTSTRAP=PASS'  
)
```

If any of these objects already exists because this block is being resumed after a partial run, inspect it instead of blindly enabling/creating a duplicate mount or auth method.

Required result:

```
Transit mount: transit/  
key:          lorawan-evidence  
type:         ecdsa-p256  
exportable:   false  
plaintext backup allowed: false
```

Create the least-privilege adapter policy:

```
path "transit/sign/lorawan-evidence/sha2-256" {  
  capabilities = ["update"]  
}  
  
path "transit/verify/lorawan-evidence/sha2-256" {  
  capabilities = ["update"]  
}
```

Enable AppRole and create the fabric-adapter role with this policy, but **do not issue or distribute a long-lived adapter SecretID yet**. The reviewed Fabric adapter runtime is not present in the repository at this checkpoint. Generate its actual SecretID only when the adapter implementation and runtime identity are ready.

Do not grant the adapter policy key create/delete/rotate, policy administration, auth administration, raw storage, Raft snapshot, or root capabilities.

Recorded Phase 20A.13A read-only preflight - 2026-08-27

PASS / CLEAN ADMIN BASELINE. Direct health was HTTP 200 on all three OpenBao nodes and authoritative Raft membership remained exactly three voters (u1c-01 leader, u1c-02/u1c-03 followers). No Phase 20A.13 administrative objects existed: transit/ absent, approle/ absent, transit/keys/lorawan-evidence absent, fabric-evidence-signer policy absent, and fabric-adapter AppRole absent. Root token was not printed, no SecretID was issued, and no Transit mutation occurred. The next allowed mutation is **only** enabling the transit/ secrets engine, followed by immediate verification before creating the evidence key.

20A.13C Create and verify the evidence signing key

After transit/ is verified present and lorawan-evidence is verified absent, create exactly one Transit key named lorawan-evidence with:


```
type=ecdsa-p256
exportable=false
allow_plaintext_backup=false
```

This is one cluster-level mutation. Immediately read the key back and require the same three properties plus `latest_version=1`. Do not rotate the key, enable deletion, create a backup, or create policy/AppRole objects in the same checkpoint. If the key already exists on a resumed run, stop and inspect it instead of overwriting or recreating it.

Recorded Phase 20A.13D signer policy creation - 2026-08-27

PASS. `fabric-evidence-signer` now exists with exactly two ACL paths: `transit/sign/lorawan-evidence/sha2-256` and `transit/verify/lorawan-evidence/sha2-256`. Both paths grant only update; no extra paths or capabilities were present. The `lorawan-evidence` key remained `ecdsa-p256`, non-exportable, plaintext backup disabled, and `latest_version=1`. AppRole remained disabled, no `fabric-adapter` role was created, and no SecretID was issued. All three OpenBao nodes remained healthy.

20A.13E Enable AppRole auth only

Enable the built-in `aprole` auth method only after the signer policy has been verified. Before mutation, require `aprole/` to be absent. After enabling it, read back the auth-method table and require `aprole/ type aprole`. Do not create the `fabric-adapter` role and do not issue a SecretID in this checkpoint. Reverify the evidence key remains version 1 and the three-node cluster remains healthy.

Recorded Phase 20A.13E existing AppRole verification - 2026-08-27

Phase 20A.13E is complete using the already-present AppRole mount. The corrected read-only gate verified `aprole/` exists, is type `aprole`, is cluster-wide (`local=false`), and uses the built-in OpenBao v2.6.2 plugin. The `fabric-adapter` role is still absent, no SecretID was issued, the `lorawan-evidence` key remains `ecdsa-p256 / non-exportable / plaintext-backup-disabled / version 1`, and `fabric-evidence-signer` still contains exactly the two approved update-only paths. No administrative mutation was executed during the closing verification.

The origin of the pre-existing AppRole mount was not established from current evidence; do not claim a cause. Do not disable/re-enable the verified mount. Continue with role creation only.

20A.13F Create and verify the fabric-adapter AppRole only

Create the role only after the existing `aprole/` mount, signer policy, and protected Transit key are verified. Required role settings are:

```
token_policies = [fabric-evidence-signer]
token_ttl = 1h
token_max_ttl = 4h
secret_id_ttl = 24h
secret_id_num_uses = 0
bind_secret_id = true
```

`bind_secret_id=true` is required so the future adapter cannot authenticate with a RoleID alone. Do not generate a SecretID in this checkpoint. After creation, read back the role and require those exact values before continuing.

Recorded Phase 20A.13G final Transit/AppRole acceptance - 2026-08-27

PASS evidence from the final read-only acceptance gate:

- authoritative Raft configuration contains exactly three voters: `u1c-01` leader, `u1c-02` follower, `u1c-03` follower
- `transit/` exists and is type `transit`

- lorawan-evidence is ecdsa-p256, exportable=false, allow_plaintext_backup=false, deletion_allowed=false, latest version 1
- fabric-evidence-signer contains exactly the two intended sign/verify paths with update only and no extra paths/capabilities
- approle/ exists as the built-in OpenBao v2.6.2 auth method with local=false
- fabric-adapter has only fabric-evidence-signer, token TTL 3600 s, max TTL 14400 s, SecretID TTL 86400 s, secret_id_num_uses=0, and bind_secret_id=true
- a RoleID exists but was not printed
- SecretID accessor count is exactly zero, so no adapter SecretID has been issued
- no administrative mutation occurred in the acceptance gate
- all three OpenBao direct health endpoints returned HTTP 200
- temporary script cleanup passed and the operator login shell survived

PHASE20A13G_FINAL_ACCEPTANCE=PASS and PHASE20A13G_EXIT=0. Phase 20A.13 is complete.

20A.14 Add the stable HAProxy KMS frontend on ulc-01 and ulc-02

20A.14A ulc-01 read-only HAProxy preflight

Before modifying `/etc/haproxy/haproxy.cfg`, verify on `ulc-01` that HAProxy is active, its current configuration validates, TCP/18200 is unused, frontend `openbao_kms` and backend `openbao_nodes` are absent, and `/etc/haproxy/openbao-ca.crt` is the same public CA as `/etc/lorawan-pki/openbao/ca.crt`. Record the current HAProxy configuration SHA-256 and current listener inventory so the later mutation has a precise before-state.

Also prove all three OpenBao APIs return HTTP 200 through direct TLS checks using SNI `openbao-kms.internal.lorawan.com` and the HAProxy CA copy. This catches certificate/health problems before HAProxy backend checks are introduced.

This checkpoint is read-only. Do not reload HAProxy, edit its configuration, create a backup, stop any OpenBao node, or modify PostgreSQL, Patroni, PgBouncer, Valkey, Mosquitto, or ChirpStack.

Only after this preflight passes should 20A.14B create a timestamped backup, append the KMS frontend/backend to an off-path candidate, validate the candidate, install it, reload HAProxy, and verify `10.104.0.2:18200` while preserving every pre-existing listener.

Only after all three OpenBao nodes are initialized and unsealed, add this shape to the existing HAProxy configuration on `ulc-01` and `ulc-02`, changing only the local bind IP:

```
frontend openbao_kms
  mode tcp
  bind <THIS_APP_PRIVATE_IP>:18200
  default_backend openbao_nodes

backend openbao_nodes
  mode tcp
  option httpchk
  http-check send meth GET uri /v1/sys/health?standbyok=true ver HTTP/1.1 hdr Host openbao-
kms.internal.lorawan.com
  http-check expect status 200

  server openbao-1 10.104.0.2:8200 check check-ssl verify required ca-file /etc/haproxy/openbao-ca.crt
check-sni openbao-kms.internal.lorawan.com
  server openbao-2 10.104.0.4:8200 check check-ssl verify required ca-file /etc/haproxy/openbao-ca.crt
check-sni openbao-kms.internal.lorawan.com
  server openbao-3 10.104.0.8:8200 check check-ssl verify required ca-file /etc/haproxy/openbao-ca.crt
check-sni openbao-kms.internal.lorawan.com
```


Application traffic remains raw TLS pass-through. check-ssl applies TLS to the HAProxy health connection; do not add ssl to the backend server data path and accidentally wrap already-TLS client traffic in another TLS layer.

For each HAProxy node:

```
sudo haproxy -c -V -f /etc/haproxy/haproxy.cfg
sudo systemctl reload haproxy
sudo systemctl is-active haproxy
sudo ss -lntp | grep -E ':18200\b'
```

Do one application node at a time. Do not restart or modify PostgreSQL, Patroni, PgBouncer, Valkey, Mosquitto, or ChirpStack as part of this change.

Recorded Phase 20A.14A ulc-01 HAProxy preflight - 2026-08-27

PASS. ulc-01 (10.104.0.2) HAProxy 2.8.16-0ubuntu0.24.04.3 was active and enabled. The current /etc/haproxy/haproxy.cfg validated cleanly with SHA-256 4b36b3b0b17a8ac438d758dcec291e2f4878c66da090b60e8d07e9003e900808, mode 0644, owner root:root. Existing runtime listeners were recorded as 10.104.0.2:15432, :15433, :16379, :18883, :8883, plus public 10.15.0.5:443 and :8883. openbao_kms and openbao_nodes were absent, TCP 18200 was free in both runtime and config, and /etc/haproxy/openbao-ca.crt matched the staged OpenBao CA with SHA-256 18a8d9960b5a0bc0476e64628bdac0e00069aeae6b6ec7f0c95324fda119af6d. TLS health checks to all three OpenBao APIs using the HAProxy CA and service SNI returned HTTP 200. The HAProxy config hash was unchanged after the gate; no backup, reload, or service mutation occurred. PHASE20A14A_ULC01_PREFLIGHT=PASS, PHASE20A14A_EXIT=0, and the login shell survived.

20A.14B ulc-01 rollback-safe KMS frontend rollout

On ulc-01 only: re-prove the Phase 20A.14A config hash and free 18200, create a timestamped root-only backup, build an off-path candidate by appending only the openbao_kms frontend and openbao_nodes backend, validate the candidate before installation, then install and reload HAProxy. After reload, require all pre-existing listeners plus exactly 10.104.0.2:18200, service active/enabled, TLS hostname verification through :18200, and HTTP 200 from /v1/sys/health?standbyok=true. If installed validation, reload, listener preservation, or KMS reachability fails, restore the recorded backup and reload the original configuration before stopping for review. Do not touch ulc-02 until this gate passes.

Recorded Phase 20A.14B ulc-01 HAProxy KMS rollout - 2026-08-27

PASS. ulc-01 re-proved the exact preflight HAProxy baseline SHA-256 4b36b3b0b17a8ac438d758dcec291e2f4878c66da090b60e8d07e9003e900808, free private TCP/18200, matching OpenBao CA copy, and direct TLS health HTTP 200 from all three OpenBao backends. A validated off-path candidate added only frontend openbao_kms bound to 10.104.0.2:18200 and backend openbao_nodes with TLS-verifying HAProxy health checks; application traffic remained raw TLS pass-through. The original config was preserved at /etc/haproxy/phase20a14b-20260827T080548Z/haproxy.cfg.before-openbao with the same original SHA-256. The installed config SHA-256 became 31d17ede04a05be0b812de3eb602e18656880a7dbd2fc718908d2b63eb7bcf47 and passed installed/final validation. HAProxy reloaded cleanly and stayed active/enabled. All seven existing listeners remained present (10.104.0.2:15432, :15433, :16379, :18883, :8883, plus 10.15.0.5:443 and :8883) and exactly one new private listener appeared at 10.104.0.2:18200. TLS hostname verification through the stable KMS path passed immediately, six repeated health probes returned HTTP 200, all three direct OpenBao backends remained HTTP 200 after reload, and rollback was not needed. ulc-02 was not modified. PHASE20A14B_ULC01=PASS, PHASE20A14B_EXIT=0, and the login shell survived.

20A.14C ulc-02 read-only HAProxy preflight

Before any ulc-02 HAProxy change, repeat the same read-only boundary used on ulc-01: require HAProxy active/enabled and current config valid, record the exact current config SHA-256 and listener inventory, require openbao_kms / openbao_nodes absent, require 10.104.0.4:18200 free in both runtime and config, verify /etc/haproxy/openbao-ca.crt exactly matches the OpenBao runtime CA, prove all three OpenBao direct TLS health endpoints return HTTP 200 using the service hostname, and prove the HAProxy config SHA-256 is unchanged at the end. This checkpoint is read-only; do not create a backup, install a candidate, reload HAProxy, or touch ulc-01.

Recorded Phase 20A.14C ulc-02 HAProxy preflight - 2026-08-27

PASS. ulc-02 remained read-only with HAProxy 2.8.16-0ubuntu0.24.04.3 active/enabled and valid current config SHA-256 30bdeef9cc99f574d75be9c33fd86359198cec001946eb2d542ebee1b891cf3. Existing runtime listeners were 10.104.0.4:15432, 10.104.0.4:15433, 10.104.0.4:16379, 10.104.0.4:18883, 10.104.0.4:8883, 10.15.0.7:443, 10.15.0.7:8883, and 127.0.1.1:15432. openbao_kms / openbao_nodes were absent, TCP/18200 was free, /etc/haproxy/openbao-ca.crt matched the OpenBao runtime CA, the already-deployed ulc-01:18200 stable KMS path returned HTTP 200, and all three direct OpenBao TLS health checks returned HTTP 200. The config hash was unchanged after the gate; no reload, backup, or service mutation occurred.

20A.14D ulc-02 rollback-safe KMS frontend rollout

Use the exact Phase 20A.14C baseline SHA-256

30bdeef9cc99f574d75be9c33fd86359198cec001946eb2d542ebee1b891cf3. Preserve every observed listener, including 127.0.1.1:15432. Build and validate an off-path candidate, create a timestamped rollback backup, install only the openbao_kms / openbao_nodes blocks with private bind 10.104.0.4:18200, gracefully reload HAProxy, and prove the old listener set plus the new KMS listener. Verify both ulc-01:18200 and ulc-02:18200 return TLS-verified HTTP 200 and all three direct OpenBao backends remain healthy. On any post-install failure, restore the exact baseline backup and reload HAProxy. Do not modify ulc-01 or any PostgreSQL, Patroni, PgBouncer, Valkey, Mosquitto, or ChirpStack configuration.

20A.15 Local service-name mapping for future adapters

Until private DNS provides the record, map the stable KMS verification name locally on each future adapter host:

```
ulc-01: openbao-kms.internal.lorawan.com -> 10.104.0.2
ulc-02: openbao-kms.internal.lorawan.com -> 10.104.0.4
```

This intentionally sends each adapter to the HAProxy instance on its own host while preserving one TLS verification name.

Do not map the service name directly to one OpenBao member; that would bypass the HAProxy routing layer.

20A.16 Normal-path acceptance — no failure injection

From each application node, verify TLS through its own local HAProxy frontend:

```
openssl s_client \
  -connect <THIS_APP_PRIVATE_IP>:18200 \
  -servername openbao-kms.internal.lorawan.com \
  -CAfile /etc/haproxy/openbao-ca.crt \
  -verify_hostname openbao-kms.internal.lorawan.com \
  -brief </dev/null
```

Then call:

```
GET https://openbao-kms.internal.lorawan.com:18200/v1/sys/health?standbyok=true
```

and require HTTP 200.

Finally create one short-lived commissioning token carrying only fabric-evidence-signer and prove the exact stable endpoint. Run on ulc-01; --add-host keeps hostname verification while forcing the test client to the local HAProxy 10.104.0.2:18200 without modifying host DNS:

```
sudo -v && (
set -euo pipefail

OPENBAO_IMAGE='docker.io/openbao/openbao@sha256:11fd73a2102cda9c55d5d881a8c3210303146a7ec1e8ac76f526e175c6d24
641'
ROOT_TOKEN="$(sudo python3 -c 'import json; print(json.load(open("/root/lorawan-openbao-bootstrap/init.json",
"r", encoding="utf-8"))["root_token"])'")"
trap 'unset ROOT_TOKEN COMMISSION_TOKEN SIGNATURE INPUT_B64' EXIT

TOKEN_JSON="$(sudo docker exec \
-e BAO_ADDR=https://10.104.0.2:8200 \
-e BAO_CACERT=/openbao/tls/ca.crt \
-e BAO_TOKEN="$ROOT_TOKEN" \
openbao bao token create \
-policy=fabric-evidence-signer -no-default-policy -ttl=15m -orphan -format=json)"

COMMISSION_TOKEN="$(printf '%s' "$TOKEN_JSON" | python3 -c 'import json,sys; print(json.load(sys.stdin)
["auth"]["client_token"])'")"
unset TOKEN_JSON ROOT_TOKEN

INPUT_B64="$(printf '%s' 'openbao-evidence-kms-self-test' | base64 -w0)"

SIGNATURE="$(sudo docker run --rm --network host \
--add-host openbao-kms.internal.lorawan.com:10.104.0.2 \
-v /etc/haproxy/openbao-ca.crt:/openbao/tls/ca.crt:ro \
-e BAO_ADDR=https://openbao-kms.internal.lorawan.com:18200 \
-e BAO_CACERT=/openbao/tls/ca.crt \
-e BAO_TOKEN="$COMMISSION_TOKEN" \
--entrypoint bao "$OPENBAO_IMAGE" \
write -field=signature transit/sign/lorawan-evidence/sha2-256 \
input="$INPUT_B64" prehashed=false marshaling_algorithm=asn1)"

test -n "$SIGNATURE"

VALID="$(sudo docker run --rm --network host \
--add-host openbao-kms.internal.lorawan.com:10.104.0.2 \
-v /etc/haproxy/openbao-ca.crt:/openbao/tls/ca.crt:ro \
-e BAO_ADDR=https://openbao-kms.internal.lorawan.com:18200 \
-e BAO_CACERT=/openbao/tls/ca.crt \
-e BAO_TOKEN="$COMMISSION_TOKEN" \
--entrypoint bao "$OPENBAO_IMAGE" \
write -field=valid transit/verify/lorawan-evidence/sha2-256 \
input="$INPUT_B64" signature="$SIGNATURE" \
prehashed=false marshaling_algorithm=asn1)"

[ "$VALID" = 'true' ]

sudo docker run --rm --network host \
--add-host openbao-kms.internal.lorawan.com:10.104.0.2 \
-v /etc/haproxy/openbao-ca.crt:/openbao/tls/ca.crt:ro \
-e BAO_ADDR=https://openbao-kms.internal.lorawan.com:18200 \
-e BAO_CACERT=/openbao/tls/ca.crt \
-e BAO_TOKEN="$COMMISSION_TOKEN" \
--entrypoint bao "$OPENBAO_IMAGE" token revoke -self >/dev/null

printf 'SIGN=PASS\nVERIFY=%s\nOPENBAO_STABLE_ENDPOINT=PASS\n' "$VALID"
)
```

The commissioning token must be created with `-no-default-policy`, and the acceptance wrapper must verify its returned policy list is exactly `fabric-evidence-signer` before using it. Repeat only the TLS/health reachability check from `ulc-02`; a second cryptographic signing test is unnecessary unless the first stable-endpoint proof fails or the KMS routing/configuration changes. Do not rotate the Transit key and do not stop an OpenBao member during setup acceptance.

20A.17 Prepared PASS boundary

OpenBao infrastructure is ready for the later Fabric adapter only when all are true:

```
[x] exact OpenBao 2.6.2 immutable image digest is present on all three nodes
[x] every node uses its own private TLS key/certificate
[x] API listeners bind only 10.104.0.2/.4/.8:8200
[x] Raft listeners bind only 10.104.0.2/.4/.8:8201
[x] exactly one OpenBao cluster was initialized
[x] exactly three Raft voters exist
[x] 3/3 are initialized + unsealed
[x] one active + two usable standbys
[x] Transit is enabled
[x] lorawan-evidence is ecdsa-p256 and non-exportable
[x] fabric-evidence-signer grants only sign/verify
[x] HAProxy :18200 exists only on ulc-01/02 private addresses
[x] HAProxy health excludes sealed/uninitialized members
[x] TLS verifies as openbao-kms.internal.lorawan.com through both :18200 paths
[x] fixed harmless sign/verify returns valid=true through :18200
[x] no Fabric adapter SecretID was issued prematurely
```

Record:

```
OPENBAO_3_NODE_NORMAL_PATH=PASS
OPENBAO_PHASE15_FAILURE_TESTS=NOT_STARTED
FABRIC_ADAPTER_RUNTIME=BLOCKED_UNTIL_IMPLEMENTATION_AND_HANDOFF
```

After this boundary, leave all three members running and return to the other pre-test commissioning work. The later adapter deployment consumes this stable KMS service; it must not reconfigure OpenBao node addresses during normal operation.

20A.18 Stop conditions

Stop immediately and preserve the current state if any of these occurs:

```
an OpenBao node is accidentally initialized separately
one node certificate does not verify the shared KMS name
8200/8201 binds wildcard/public instead of the private VPC IP
fewer than three voters appear after join
an intended voter remains sealed
HAProxy marks a sealed/uninitialized backend healthy
HAProxy reload removes or changes an already-commissioned listener
Transit key is exportable or plaintext-backup capable
normal sign/verify through :18200 fails
```

Do not repair these by weakening TLS, changing the KMS name, using `-tls-skip-verify`, making the Transit key exportable, or deleting existing HA configuration.

Phase 20A.12 Shamir join-order correction

With Shamir sealing, `initialized=true` / `sealed=true` on a `retry_join` peer is a valid **join-pending** state. The node must receive threshold unseal shares before it can complete the encrypted Raft join challenge and become visible

in raft list-peers. Therefore the previous assertion requiring ulc-02 to appear as a voter before unseal was incorrect. Correct order: initialized/sealed join-pending -> submit two distinct shares -> verify unsealed -> verify committed Raft membership.

Recorded Phase 20A.12 ulc-02 join attempt 1 - 2026-08-27

INCOMPLETE JOIN / NO UNSEAL ATTEMPT. ulc-02 started cleanly with normalized Compose, valid retry_join configuration, restart count 0, and no duplicate-config warning. Its local bao status reached initialized=true, sealed=true, storage_type=raft, and sealed health HTTP 503. However, authoritative membership from the active leader still listed only ulc-01; the two-voter assertion failed before any Shamir share was submitted. After reviewing the documented Shamir join handshake, this is the expected join-pending state: ulc-02 must receive two distinct unseal shares before it can answer the encrypted join challenge and become visible in the leader's peer set. Continue by unsealing ulc-02 from ulc-01 protected bootstrap material, then require exactly two voters. Do not restart ulc-02, delete its Raft data, copy init.json to it, or start ulc-03.

Recorded Phase 20A.12 ulc-02 unseal attempt 2 - 2026-08-27

INCOMPLETE / SECOND SHARE RESET PROGRESS. ulc-02 remained in the expected join-pending state (initialized=true, sealed=true) before unseal. The leader still listed only ulc-01. The first Shamir share was accepted with progress=1; the second distinct share returned sealed=true, progress=0 instead of unsealing the node. The operator wrapper stopped immediately. Do not submit additional shares, restart ulc-02, delete its Raft data, or start ulc-03 until ulc-02 logs and current leader membership are reviewed.

Working interpretation: OpenBao documents the Shamir Raft join as a challenge/answer flow where the joining node needs enough unseal shares to decrypt the leader challenge before membership can complete; it also states that joined Shamir nodes still require manual unseal. Therefore progress=1 followed by progress=0 with sealed=true may represent completion of the join-authentication threshold followed by a reset into the normal sealed state. Treat this only as a hypothesis until the leader peer set and ulc-02 logs confirm it; do not submit another share before that read-only verification.

Recorded Phase 20A.12 post-threshold diagnostic wrapper failure - 2026-08-27

INCOMPLETE / NO REMOTE DIAGNOSTIC EXECUTED. The operator wrapper printed its local section headers and exited 0, but neither the ulc-02 read-only body nor the ulc-01 Raft-membership body produced any output. Root cause: the wrapper combined ssh -n with bash -s fed by a heredoc. ssh -n redirects SSH stdin from /dev/null, so the heredoc script was not transmitted. This changed no OpenBao, Raft, seal, container, or peer state. The correct pattern is to finish the heredoc locally, copy the diagnostic script to the target node, then invoke it by path over SSH. Do not submit another Shamir share until the corrected read-only diagnostic proves whether ulc-02 entered the Raft peer set after the first two-share threshold cycle.

Recorded Phase 20A.12 ulc-02 completed join - 2026-08-27

PASS. The corrected read-only diagnostic proved the first two-share cycle was sufficient for ulc-02 to finish both the retry_join challenge and operational unseal. Current ulc-02 seal status was initialized=true, sealed=false, progress=0, threshold 2, shares 3, storage_type=raft, with cluster name/ID present. Both private listeners 10.104.0.4:8200 and 10.104.0.4:8201 were active, restart count remained 0, and Raft files existed. Logs recorded successfully joined the raft cluster, initial nonvoter membership for ulc-02, then vault is unsealed and post-unseal setup complete. Authoritative Raft configuration from ulc-01 showed exactly two voters: ulc-01 leader and ulc-02 follower. Therefore do **not** submit a second unseal cycle to ulc-02. The second share response showing sealed=true, progress=0 was an intermediate response during the asynchronous join completion, not proof that another threshold was required.

For later retry_join peers, after the second share do not assert directly on that immediate response. Instead poll both the joining node seal-status and the leader's Raft configuration until one of these bounded outcomes occurs:

(1) peer is committed and sealed=false -> PASS; (2) peer is committed but remains sealed=true after the join settles -> review before any additional shares; (3) peer never appears in Raft -> inspect logs before more shares.

Phase 20A.12C ulc-03 join execution rule

Use the proven ulc-02 sequence for ulc-03, with one correction: after the second Shamir share, do not assert that the immediate `/v1/sys/unseal` response must already show `sealed=false`. The ulc-02 evidence showed that response can still be an intermediate join state while `retry_join` is asynchronously completing. After two distinct shares are submitted, poll both (a) ulc-03 `/v1/sys/seal-status` and (b) the active leader `/v1/sys/storage/raft/configuration`. PASS only when ulc-03 is `initialized=true`, `sealed=false`, the leader lists exactly ulc-01, ulc-02, ulc-03, all are voters, and exactly one leader exists. If membership commits but ulc-03 remains sealed after the bounded settle window, stop and review before submitting any additional share.

Recorded Phase 20A.12C ulc-03 completed join - 2026-08-27

PASS. ulc-01 and ulc-02 were healthy before ulc-03 start. ulc-03 began from an empty Raft data directory with two `retry_join` stanzas, validated OpenBao v2.6.2 configuration, and normalized Compose to command: `["server"]`. After start it reached join-pending `initialized=true`, `sealed=true`, `storage_type=raft`, HA enabled, and health HTTP 503. Two distinct Shamir shares were submitted from ulc-01 without copying bootstrap material to ulc-03. The immediate second-share response still showed `sealed=true`, `progress=0`, matching the ulc-02 asynchronous join behavior; bounded polling then proved ulc-03 became `sealed=false` and the authoritative Raft configuration contained exactly three voters: ulc-01 leader, ulc-02 follower, ulc-03 follower. ulc-03 returned health HTTP 200, listeners `10.104.0.8:8200` and `10.104.0.8:8201` were present, restart count remained 0, and no bootstrap file existed on ulc-03. `PHASE20A12_ULC03=PASS`; `PHASE20A12_ULC03_EXIT=0`; `LOGIN_SHELL_SURVIVED=YES`.

Recorded Phase 20A.13B Transit mount enable - 2026-08-27

PASS. `transit/` was absent before mutation, enabled exactly once on the active OpenBao cluster, and verified as a transit secrets-engine mount. The later Phase 20A.13 objects remained absent: no `lorawan-evidence` key, no `fabric-evidence-signer` policy, no AppRole auth method, no `fabric-adapter` role, and no SecretID issuance. All three OpenBao members returned health HTTP 200 after the mount mutation. `PHASE20A13B_TRANSIT=PASS`; `PHASE20A13B_EXIT=0`; `LOGIN_SHELL_SURVIVED=YES`.

Recorded Phase 20A.13B accidental rerun refusal - 2026-08-27

SAFE REFUSAL / NO MUTATION. Phase 20A.13B was rerun after Transit had already been enabled. The precondition check observed `transit/` already present with type `transit` and aborted before executing `bao secrets enable`, key creation, policy creation, AppRole enablement, role creation, or SecretID issuance. Treat the nonzero exit as expected guardrail behavior; the previously recorded Phase 20A.13B PASS remains authoritative. Continue with Phase 20A.13C key creation.

Recorded Phase 20A.13C evidence key creation - 2026-08-27

PASS. `lorawan-evidence` was created exactly once under `transit/` and verified as `ecdsa-p256`, `exportable=false`, `allow_plaintext_backup=false`, `latest_version=1`, and `deletion_allowed=false`. The 3-node OpenBao cluster remained healthy. No signer policy, AppRole auth method, `fabric-adapter` role, SecretID, or key rotation was created/executed in this checkpoint.

20A.13D Create and verify the least-privilege signer policy

Create only `fabric-evidence-signer` after `lorawan-evidence` is verified. The policy must contain exactly these two paths, each with only update capability:

```
path "transit/sign/lorawan-evidence/sha2-256" {
  capabilities = ["update"]
}
```



```
}
path "transit/verify/lorawan-evidence/sha2-256" {
  capabilities = ["update"]
}
```

Reject wildcards or any key create/delete/rotate, policy administration, auth administration, raw storage, snapshot, sudo, root, read, list, create, delete, patch, or other capabilities. Verify the stored policy after writing it. Do not enable AppRole or issue a SecretID in this checkpoint.

Recorded Phase 20A.13E precondition refusal - 2026-08-27

SAFE REFUSAL / NO APPROLE MUTATION BY THIS RUN. The Phase 20A.13E guard found `approle/` already present before the intended enable operation and exited before attempting `sys/auth/approle`. The returned mount reports `type=approle`, built-in OpenBao v2.6.2, `local=false`, and supported status. Do not disable or re-enable it. Before Phase 20A.13F, perform a read-only check that the existing mount is healthy and that `auth/approle/role/fabric-adapter` is still absent. The cause/timing of the prior AppRole enablement is not proven by this output alone.

Recorded Phase 20A.13E-R policy verifier schema mismatch - 2026-08-27

The read-only AppRole resume check verified the existing `approle/` mount as the built-in OpenBao v2.6.2 auth method with `local=false`, verified `fabric-adapter` is still absent, and reverified the protected `lorawan-evidence` key. The check then failed only because the verifier assumed the ACL-policy API response contained `data.rules`; this field was not present in the returned shape. Treat this as a verifier/schema mismatch only. No administrative mutation was executed. The cause is now confirmed: `GET /v1/sys/policies/acl/:name` returns the ACL body in `data.policy` in the normal API envelope, while the legacy `/sys/policy/:name` form uses `rules`; the failed verifier incorrectly expected `data.rules`. Future checks for this endpoint must read `data.policy` (or use `bao policy read`). Do not disable/re-enable AppRole, recreate the key, or overwrite the policy. Re-run only a read-only policy inspection using the actual response shape before advancing to 20A.13F.

Recorded Phase 20A.13F fabric-adapter AppRole creation - 2026-08-27

Observed PASS:

- AppRole mount precondition passed.
- `fabric-evidence-signer` policy precondition passed.
- `lorawan-evidence` key remained `ecdsa-p256`, `exportable=false`, `allow_plaintext_backup=false`, version 1.
- `fabric-adapter` role was absent before creation and was created exactly once.
- role read-back: policy `fabric-evidence-signer`; token TTL 3600; token max TTL 14400; SecretID TTL 86400; SecretID uses 0; `bind_secret_id=true`.
- RoleID exists but was not printed.
- no SecretID was issued.
- all three OpenBao nodes remained HTTP 200 with `standbyok=true`.

Phase 20A.13F: **PASS**.

20A.13G Final read-only Transit/AppRole acceptance

Before Phase 20A.14 HAProxy, run one final read-only gate that proves the whole Phase 20A.13 state without issuing credentials:

- exact 3-voter Raft membership and one leader;
- `transit/` exists;

- lorawan-evidence remains ecdsa-p256, non-exportable, plaintext backup disabled, deletion disabled, version 1;
- fabric-evidence-signer contains exactly the sign and verify paths with only update;
- approle/ is the built-in cluster-wide AppRole mount;
- fabric-adapter has only fabric-evidence-signer, token TTL 1h, max 4h, SecretID TTL 24h, secret_id_num_uses=0, and bind_secret_id=true;
- RoleID exists but is not printed;
- list SecretID accessors for fabric-adapter and require zero accessors;
- all three OpenBao nodes remain healthy.

Do not create a SecretID in this gate.

Recorded Phase 20A.14D ulc-02 HAProxy KMS rollout - 2026-08-27

PASS. ulc-02 started from the exact preflight HAProxy configuration SHA-256

30bdeef9cc99f574d75be9c33fd86359198cec001946eb2d542ebee1b891cf3. The rollout re-proved the existing ulc-01:18200 KMS path and all three direct OpenBao TLS health endpoints before mutation, built and validated an off-path candidate, preserved a rollback copy at /etc/haproxy/phase20a14d-20260827T082022Z/haproxy.cfg.before-openbao, and installed the validated candidate.

The final ulc-02 HAProxy configuration SHA-256 is

7b2f1520bb07d10438f65cc08936bc7a331fd685ae23b3974e514def1f0a3f46. HAProxy reloaded successfully and remained active/enabled. Every pre-existing listener was preserved, including the node-specific 127.0.1.1:15432 PostgreSQL bind, and the new KMS frontend exists only at 10.104.0.4:18200. TLS hostname verification through the new frontend returned HTTP 200 immediately. Three repeated probes through each of the ulc-01 and ulc-02 KMS frontends returned HTTP 200, and all three direct OpenBao backends remained HTTP 200. No rollback was required and ulc-01 HAProxy configuration was not modified by this step.

Phase 20A.14 status: COMPLETE / PASS. The stable OpenBao KMS HAProxy frontend is now present on ulc-01:18200 and ulc-02:18200 only, with application TLS preserved end-to-end through TCP pass-through.

Continuation checkpoint / intentional pause. Server OpenBao work is paused here while gateway work resumes. When returning to this server track, continue with **Phase 20A.15 local service-name mapping**, then **Phase 20A.16 normal-path acceptance**. Do not repeat Phase 20A.14 unless the HAProxy configuration or KMS routing changes.

Phase 20A.15 local KMS service-name mapping - 2026-08-27

PASS. The complete operator wrapper ran from ulc-03 and first re-proved SSH identity for both application nodes using the existing root-controlled deployment key. On ulc-01, /etc/hosts had no prior active mapping for openbao-kms.internal.lorawan.com; a rollback copy was created at /root/phase20a15-ulc-01-20260827T131136Z/hosts.before, then the stable name was mapped exactly to the local HAProxy address 10.104.0.2. getent ahostsv4 resolved only 10.104.0.2, OpenSSL hostname verification through :18200 passed, the normal resolver-path OpenBao health request returned HTTP 200, and HAProxy configuration remained valid. On ulc-02, the same guarded sequence created rollback copy /root/phase20a15-ulc-02-20260827T131138Z/hosts.before, mapped the shared KMS name exactly to local HAProxy 10.104.0.4, resolved only 10.104.0.4, passed TLS hostname verification, returned HTTP 200 through the normal resolver path, and left HAProxy valid. Both per-node gates passed, the enclosing operator returned PHASE20A15_OPERATOR_EXIT=0, and the ulc-03 login shell survived.

Decision: **Phase 20A.15 is COMPLETE / PASS.** The two future adapter/application hosts now use one TLS verification name while each resolves it to its own HAProxy KMS frontend; no mapping points directly at an OpenBao Raft member. Next: Phase 20A.16 normal-path acceptance. Run TLS/health acceptance from both

application nodes, then perform exactly one short-lived fabric-evidence-signer Transit sign/verify test through the stable ulc-01 HAProxy endpoint. Do not rotate the Transit key and do not inject failure during this acceptance.

Recorded Phase 20A.16 normal-path acceptance - 2026-08-27

PASS. The entire operator workflow ran from ulc-03 and re-proved SSH identity to both application nodes. On ulc-01, openbao-kms.internal.lorawan.com resolved only to local HAProxy 10.104.0.2; on ulc-02 it resolved only to local HAProxy 10.104.0.4. Both HAProxy configurations remained valid, both private :18200 listeners were present, TLS hostname verification passed through the normal resolver path, and /v1/sys/health?standbyok=true returned HTTP 200 through both stable endpoints.

Exactly one cryptographic acceptance test then ran on ulc-01. The protected bootstrap file remained 0600 root:root; the root token was loaded without terminal output; Transit key lorawan-evidence was version 1 before the test. A 15-minute orphan commissioning token was created with -no-default-policy, and its returned policy list was exactly fabric-evidence-signer. A fixed harmless input signed successfully through https://openbao-kms.internal.lorawan.com:18200, verification returned true, and the Transit key remained version 1 afterward. The temporary token was explicitly revoked and final KMS health remained HTTP 200. No OpenBao member was stopped and no failure injection or key rotation occurred. PHASE20A16_NORMAL_PATH_ACCEPTANCE=PASS; PHASE20A16_OPERATOR_EXIT=0; ULC03_LOGIN_SHELL_SURVIVED=YES.

Recorded Phase 20A.17 prepared PASS boundary - 2026-08-27

COMPLETE / PASS. All prepared-boundary checklist items above are now backed by recorded execution evidence. The OpenBao infrastructure is ready to be consumed by the later Fabric adapter without further node-address, Raft, Transit-key, signer-policy, or HAProxy KMS reconfiguration. Keep all three members running. OPENBAO_3_NODE_NORMAL_PATH=PASS; OPENBAO_PHASE15_FAILURE_TESTS=NOT_STARTED; FABRIC_ADAPTER_RUNTIME=BLOCKED_UNTIL_IMPLEMENTATION_AND_HANDOFF.

The adapter block above records the historical 2026-08-27 boundary. It has now been superseded by the commissioning update below.

Fabric-adapter runtime commissioning update - 2026-09-02

The implementation half of that historical block is now closed. An immutable Fabric-adapter image is deployed on both ulc-01 and ulc-02 in enabled=false fail-closed standby. A dedicated database credential was rotated into protected custody, the authoritative ten-role SCRAM userlist was refreshed on all three PgBouncer nodes by reload-in-place with unchanged PIDs, and fabric_adapter authenticated through all three physical PgBouncer endpoints using sslmode=verify-full.

Separate OpenBao AppRole SecretIDs were issued for fabric-adapter-ulc-01 and fabric-adapter-ulc-02, installed root:65532 mode 0440, and each identity independently passed AppRole login plus Transit sign/verify and tamper rejection through the stable :18200 HA endpoint. Both hosts now have /etc/lorawan-cloud/gateway-evidence/fabric-adapter.env staged 0600 root:root with real database/OpenBao settings and explicit placeholders only for the external Fabric Gateway/TLS/MSP/channel/chaincode/contract/function handoff.

fabric-adapter-enable-preflight.sh correctly refuses activation while those placeholders remain, and both adapters stay healthy standby. The current boundary is therefore FABRIC_ADAPTER_RUNTIME=READY_INTERNAL_EXTERNAL_FABRIC_HANDOFF_PENDING, not missing implementation.

Supporting runbook: 20-openbao-and-fabric-adapter.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

20. OpenBao + Fabric Adapter for the Tiny HA POC

Status: ULC-01 TASK-37 EXTERNAL QUALIFICATION PASS. Three-node OpenBao, audit, telemetry.fabric_outbox, the Fabric-adapter runtime, database authentication, and the HRC Task 37 handoff are commissioned. HRC hrc-evidence v1.1 / sequence 2 is live-qualified and the frozen API is `CreateSourceBoundAnchor(SourceRecordID, sourceType, producer, producedAt, schemaVersion)` with exact payload only in transient `hrc.exact_payload`, followed by `QuerySourceBoundAnchor(SourceRecordID)` and `VerifySourceBoundDigest(SourceRecordID, observedDigest)`. On 2026-09-08 ULC-01 completed the explicitly authorized one-record qualification for outbox 1960 / SourceRecordID fdadcf14-741e-4293-ad38-39ba89369ea7. The exact 2828-byte payload SHA-256 was 827e443857118c0f922c2f03e66f950a79092c74d8d8b947d489ace43d157b51; the resulting Fabric transaction ID is dd11910c1e7c28edd11334a03cce10d80d574692733458c9059d2775be5131b8 and the stored HRC record ID is hrc-record-ad1258ae5ee243c01d1e091c684df1c950589bcbcd820d736a257f8cd052bfb. The one-shot runner reached confirmed only after durable prepared-transaction/commit-status state, authoritative commit success, `QuerySourceBoundAnchor` field matching, and `VerifySourceBoundDigest=MATCH`; the final outbox state is confirmed, attempts=1. Two defects encountered before the successful write were corrected without producing a duplicate Fabric proposal: the common outbox `finish()` helper now types nullable SQL parameters explicitly, and the expired ULC-01 OpenBao SecretID was refreshed using the current active OpenBao member (10.104.0.4) after audit evidence showed that SecretID issuance against a standby can fail persistence with read-only storage. The existing AppRole role/policy, Transit key, RoleID, Fabric identity, channel, chaincode, and source binding were not changed. As re-verified on 2026-09-09, Adapter-1 on ULC-01 is now the continuous production writer with `FABRIC_ADAPTER_ENABLED=true`; the authoritative activation preflight passes and 325 post-activation outbox records have reached confirmed on Fabric. The live image is sha256:d12e73ae24f7823730b6632fe8209e844c0d6125bd6a9c13e9d75cc330664f74 with service-binary SHA-256 a53990277d9032e60a9f0bfb52619a21e8c999a3573477dbfb3efe6ca6fb9591. The one-shot qualification container was removed; temporary SSH authorization and secret-bearing qualification staging were removed. Adapter-2 on ULC-02 remains `FABRIC_ADAPTER_ENABLED=false` and must stay deferred until the lease-renewal/ownership-fencing HA gate is proven. Do not rerun candidate 1960, use the old K3s-only 10.43.25.198:7051 endpoint, or use legacy Fabric APIs. Future AppRole credential rotation must respect the role's 24-hour SecretID TTL and issue against the current active OpenBao member.

20.1 Goal

Keep the future security/integration **shape** in the POC without creating another database server.

Our side owns:

```
lorawan_telemetry.fabric_outbox
Fabric adapter-1/2
OpenBao Transit KMS
integration credentials
submission/reconciliation logic
```

The external Fabric team owns its Fabric network, Gateway endpoint, organizations, channel, and chaincode.

20.2 Placement

```
ha-01
  OpenBao-1
  Fabric adapter-1

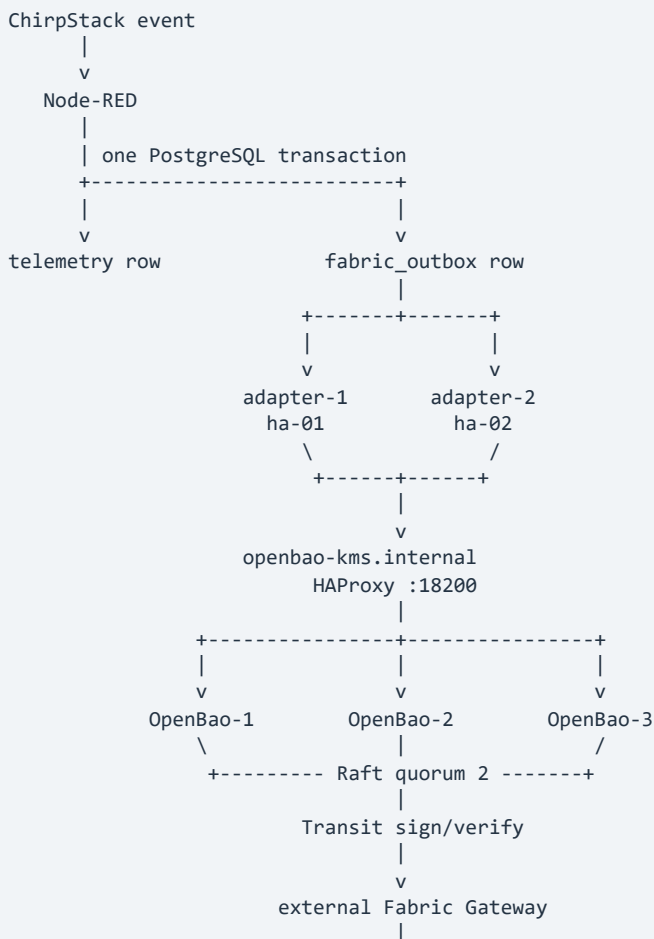
ha-02
  OpenBao-2
  Fabric adapter-2

ha-03
  OpenBao-3
```

The outbox is not on ha-03. It is a table in `lorawan_telemetry` on the **three-member Patroni PostgreSQL cluster**.

That means a PostgreSQL primary change automatically carries the outbox with the rest of the database state.

20.3 Architecture



```

      v
channel + chaincode
      |
commit status / tx ID
      |
      v
update fabric_outbox

```

20.4 Database path

Both adapters use their host-local database route:

```

Fabric adapter
-> PgBouncer :6432
-> HAProxy :15432
-> current Patroni primary :5432
-> lorawan_telemetry
-> telemetry.fabric_outbox

```

Do not give the adapters a fixed PostgreSQL node IP.

20.4A Prepared OpenBao-only parallel subphase

The OpenBao infrastructure itself has no physical-gateway dependency. While Phase 11 is compiling, the three-node KMS may be commissioned independently by following [20A. OpenBao Three-Node HA Deployment Runbook](#). This does **not** advance the Fabric adapter or full Phase 20 pass: outbox integration, adapter deployment, and one real Fabric commit still wait for their own prerequisites.

The prepared cloud pin is OpenBao 2.6.2 at OCI index digest
sha256:11fd73a2102cda9c55d5d881a8c3210303146a7ec1e8ac76f526e175c6d24641 (linux/amd64 manifest
sha256:e29524ba7c3f20d01f562c481e3eccbad6c91df45a2f2531433da4951e408cff). Normal-path setup ends after 3/3 Raft health, private TLS, Transit key/policy, HAProxy :18200, and one harmless sign/verify. Member-loss and quorum-loss remain Phase 15 tests.

20.5 OpenBao POC cluster

Use [../fabric-attestation/01-deploy-openbao-kms.md](#) as the detailed OpenBao command/config reference, but apply this **cloud POC mapping** instead of copying its larger production example blindly:

```

OpenBao node count:      3, not 5
OpenBao-1:               ha-01
OpenBao-2:               ha-02
OpenBao-3:               ha-03
API:                     private TLS :8200
Raft/cluster:             private :8201
adapter stable endpoint:  openbao-kms.internal.lorawan.com:18200 on ha-01/02 HAProxy
server SAN:               openbao-kms.internal.lorawan.com + node private name/IP

```

Bootstrap step-by-step:

1. install the same pinned OpenBao version/config pattern on all three hosts;
2. configure Integrated Storage/Raft with each node's unique node_id, api_addr, and cluster_addr;
3. install node-specific TLS key/certificate plus the common internal CA;
4. start **OpenBao-1 only** and initialize the cluster exactly once;
5. move the recovery/unseal material immediately to its protected off-node location;

6. unseal OpenBao-1 and record the initial Raft state;
7. start OpenBao-2 and join it to the existing Raft cluster—never initialize it separately;
8. unseal OpenBao-2 and verify it appears in `bao operator raft list-peers`;
9. repeat for OpenBao-3;
10. verify all three members are initialized, unsealed, and one is active;
11. enable/configure the Transit engine and create/verify the non-exportable lorawan-evidence key under the approved policy;
12. only now enable the HAProxy :18200 frontend from [07-haproxy-and-pgbouncer.md](#);
13. call `/v1/sys/health?standbyok=true` and a harmless Transit sign/verify through the stable endpoint;
14. record the healthy 3/3 Raft peer/seal state and stable-endpoint sign/verify evidence;
15. keep all three members running while proceeding to adapters.

Stop here if any node was initialized as a separate cluster, if fewer than three intended members are visible, or if HAProxy treats a sealed node as healthy. **Do not stop an OpenBao member in setup; the 2/3 survival test belongs to Phase 15.**

Use three Integrated Storage/Raft members:

```
OpenBao-1 ha-01
OpenBao-2 ha-02
OpenBao-3 ha-03
quorum = 2
```

Private ports:

```
8200 OpenBao API/TLS
8201 OpenBao Raft
18200 HAProxy stable adapter-facing KMS endpoint
```

The pre-test setup must demonstrate:

```
3/3 healthy/unsealed
Transit key exists and is non-exportable
stable :18200 endpoint is healthy
adapter identity can sign/verify through the stable endpoint
fixed canonicalization/digest vectors pass
```

The one-member-loss / 2-of-3 KMS survival experiment is reserved for Phase 15.

The evidence signing key stays non-exportable. An adapter never falls back to a local signing key.

20.6 Outbox

Create the outbox inside the Timescale-enabled `lorawan_telemetry` database on the Patroni cluster. Keep `telemetry.fabric_outbox` as an **ordinary PostgreSQL table** with the documented lease, index, permission, and immutability rules; do not convert the work queue itself into a hypertable. The telemetry event tables in the same database remain Timescale hypertables.

Required POC behavior:

```
BEGIN
  write telemetry using stable event identity
  create selected outbox row
COMMIT
```

Then the adapters process the outbox asynchronously.

Why: Fabric or KMS downtime should make the queue wait, not make sensor telemetry disappear.

20.7 Fabric writer ownership and HA target

```
ULC-01 / adapter-1 -> ENABLED production writer
ULC-02 / adapter-2 -> INSTALLED but FABRIC_ADAPTER_ENABLED=false
```

Both candidates have distinct worker identities, but lease-based row claiming alone is not sufficient proof that two processes may safely own an external Fabric side effect. The commissioned normal path therefore keeps only ULC-01 enabled. ULC-02 must remain fail-closed until the separate ownership/fencing and failover acceptance gate proves that at most one writer can submit/reconcile externally at a time.

A timeout after Fabric submission enters durable reconciliation using the already-prepared transaction and signed commit-status request; it is not permission to generate a fresh transaction. ULC-01 loss and deliberate ULC-02 takeover remain a later HA acceptance test.

20.8 External Fabric handoff

For a new environment, follow [../fabric-attestation/01-collect-external-fabric-handoff.md](#) and collect the real values from the other team. For this commissioned HRC environment, the handoff is already complete: private Gateway 10.104.0.7:7051, TLS name peer1.hrc.local, MSP HrcMSP, channel hrc-channel, chaincode hrc-evidence, empty/default contract, and source identity lorawan-gateway-evidence.

```
<FABRIC_GATEWAY_ENDPOINT>
<FABRIC_GATEWAY_PORT>
<FABRIC_TLS_SERVER_NAME>
<FABRIC_MSP_ID>
<FABRIC_CHANNEL_NAME>
<FABRIC_CHAINCODE_NAME>
submit function
query function
commit-status behavior
CA certificate
client identity
```

Do not deploy a Fabric test network on these three Droplets.

20.8A Historical server-first security readiness — commissioning complete

The adapter implementation gate below does **not** prevent independent preparation that requires no gateway hardware and no Fabric worker runtime.

The block below records the pre-activation gate that was used before Fabric credentials existed. It is retained for rebuild/recovery context and is not a statement that credentials are still absent:

```
OpenBao 3/3 healthy/unsealed through the stable :18200 endpoint
OpenBao audit-device state explicitly inspected and a reviewed audit path commissioned
fabric-adapter SecretID accessor count remains zero while no runtime exists
Node-RED selected-event atomic telemetry + outbox enqueue proven with the isolated synthetic fixture
frozen telemetry-attestation-v1 canonical-byte fixture hash reproduces exactly
expected v1 SHA-256 = c2952e8cddc7f39a17522cb49dd3292c9af75c00fdc37172f74bb3dc955f3a5c
```

The frozen v1 canonical string is already committed in ../integrations/hyperledger-fabric/04-data-contract-and-chaincode.md. Reproducing its exact SHA-256 is safe now and proves the byte fixture has not drifted. A true RFC 8785 **canonicalizer implementation** still belongs to the reviewed adapter/runtime and must pass the same vector at startup; hashing the already-frozen canonical string is not a substitute for that implementation test.

OpenBao auditing is part of the **signing** trust boundary. The Fabric security/incident-response manuals depend on KMS audit evidence to establish a possible unauthorized-signing window. The current cloud execution history does not record a commissioned audit device, so inspect that state and close the gap before releasing any signing workload credential. This does **not** block the gateway/security evidence ingest, collector, verifier, trusted-decoder, storage, or database implementation because those services hold no OpenBao/Fabric signing authority. Keep audit storage protected and separate from Raft data, use bounded host-side rotation, and avoid casually disabling/re-enabling an audit device because that destroys its existing salt/HMAC continuity.

Current commissioned state: the pre-activation gate is complete. ULC-01 is the enabled production writer and uses the commissioned least-privilege OpenBao/Fabric identity; ULC-02 retains protected standby material but remains FABRIC_ADAPTER_ENABLED=false. Do not copy one worker's SecretID or Fabric private identity to the other host, and do not enable ULC-02 until the HA ownership/fencing gate passes.

20.8B Adapter implementation readiness gate

The Fabric adapter implementation, immutable runtime image, HRC Task 37 handoff, qualification transaction, and ULC-01 continuous-writer activation are commissioned. The current recorded ULC-01 image is sha256:d12e73ae24f7823730b6632fe8209e844c0d6125bd6a9c13e9d75cc330664f74. ULC-02 remains the deliberately write-disabled standby. The remaining Fabric infrastructure gate is ULC-02 ownership/fencing failover acceptance, not another external handoff.

Before attempting adapter-1 or adapter-2, require all of these:

```
reviewed adapter source exists
build/test procedure exists
immutable image digest exists
runtime UID/GID is known
supported Fabric SDK/version is known
outbox claim/reconcile implementation matches the documented schema
OpenBao Transit client behavior is implemented
external Fabric handoff test vector passes
```

Then follow [../fabric-attestation/02-create-outbox-and-adapter.md](#), substituting the cloud endpoints:

```
lab DB host telemetry-db:5432
-> cloud pgbouncer.internal.lorawan.com:6432 / lorawan_telemetry

lab OpenBao service openbao:8200
-> cloud https://openbao-kms.internal.lorawan.com:18200

one lab adapter
-> ULC-01 adapter-1 enabled in production
-> ULC-02 adapter-2 installed with a separate protected identity but write-disabled until HA fencing acceptance
```

The adapter implementation/image gate is now **PASS**. Keep the workers in fail-closed standby until the external Fabric Gateway endpoint/TLS identity, MSP client identity, channel/chaincode/contract/function values are installed and fabric-adapter-enable-preflight.sh passes. Until that handoff is complete, do not claim a real ledger transaction or Fabric-adapter failover acceptance.

Gateway-integrity v2 has passed its implementation gate: reviewed ingest/collector/verifier/trusted-decoder components and the evidence storage path are commissioned, with real Gateway-01 lineage accepted. Keep

Fabric activation independent from that evidence lane; evidence readiness is not permission to release Fabric signing credentials.

20.9 Failure behavior

External Fabric unavailable

```
LoRaWAN          continues
Node-RED          continues on the current active host; standby remains stopped
PostgreSQL telemetry continues
fabric_outbox     accumulates
adapters         wait/reconcile/retry
```

One adapter lost

```
other worker remains
expired jobs can be reclaimed
outbox remains in PostgreSQL HA cluster
```

One OpenBao member lost

```
2/3 quorum remains
KMS endpoint remains usable
```

ha-03 lost

```
Node-RED A is lost; after fencing, Node-RED B on ha-02 is promoted
Grafana pauses
PostgreSQL stays available on ha-01/ha-02
lorawan_telemetry stays available
fabric_outbox stays available
adapter-1/2 stay alive
OpenBao remains 2/3
existing eligible outbox work can continue
```

This is a stronger POC shape than keeping the outbox in a single standalone telemetry database.

20.10 Minimal monitoring

For this POC, command-line evidence is enough:

```
OpenBao member/raft status
sealed/unsealed state
Transit sign/verify result
fabric_outbox counts by status
oldest pending age
adapter-1/2 logs
worker_id and lease owner
Fabric tx ID / commit status
```

Do not deploy a large monitoring stack solely for this integration proof.

20.11 POC backup

Before destructive tests keep:

```
lorawan_telemetry logical dump
OpenBao Raft snapshot or documented rebuild/recovery material appropriate to the test
OpenBao recovery/unseal material outside the runtime hosts
Fabric handoff metadata
adapter configuration/image reference
```

Production backup/retention is not what this POC is trying to certify.

20.12 Pre-test setup order

1. confirm Phase 12A telemetry + fabric_outbox atomic commit and retry safety
2. deploy OpenBao 3/3
3. prove Transit sign/verify through :18200 with the non-exportable key
4. run the fixed RFC 8785 canonicalization vector
5. calculate SHA-256 over the exact canonical UTF-8 bytes and verify the expected digest
6. validate the external Fabric handoff and TLS identity
7. complete the adapter implementation readiness gate

Steps 1-7 and the ULC-01 activation path below are already commissioned on the current build. For a future rebuild, re-run only the relevant gates. Do not treat this sequence as permission to enable ULC-02:

8. deploy/validate adapter-1 on ULC-01; keep adapter-2 on ULC-02 write-disabled
9. require immutable `finalized\_payload`; for v2 require verifier-owned `verified` state
10. build/verify the separate local RFC 8785/OpenBao evidence seal
11. compute SHA-256 over the exact immutable `finalized\_payload` bytes for HRC
12. prepare `CreateSourceBoundAnchor` with five metadata args + transient `hrc.exact\_payload`
13. persist txid, record_id, endorsed transaction bytes, and signed commit-status request before orderer submit
14. submit the already-prepared transaction
15. require authoritative commit status
16. require `QuerySourceBoundAnchor` field match + `VerifySourceBoundDigest=MATCH`
17. keep ULC-01/OpenBao/Fabric healthy; test ULC-02 takeover only under the explicit HA gate

Do **not** remove an OpenBao member, stop an adapter, or block Fabric connectivity here. Those are Phase 15 experiments.

20.13 Pass condition

Pre-test architecture/infrastructure pass

Require:

- outbox lives in lorawan_telemetry on the Patroni cluster;
- TimescaleDB remains inside that Patroni cluster;
- Node-RED atomically creates telemetry + selected outbox work;
- OpenBao is 3/3 healthy/unsealed through the stable endpoint;
- the Transit key is non-exportable;
- RFC 8785 canonicalization and SHA-256 fixed vectors pass;
- external Fabric handoff is complete and TLS identity is verifiable.

Full pre-test Fabric execution pass

Claim this **only when a reviewed adapter implementation/image exists** and all are proven on the healthy path:

- ULC-01 adapter-1 is the enabled writer and ULC-02 adapter-2 remains write-disabled until HA ownership/fencing is accepted;
- the selected event is converted to the fixed canonical evidence projection;
- the locally stored SHA-256 digest matches the exact canonical bytes;
- OpenBao signs/verifies the same canonical bytes and the versioned signature/key ID are persisted;
- a real selected event reaches valid external Fabric commit status;
- the returned Fabric tx ID/status is recorded in the outbox;
- read-only reconstruction can re-create the digest and verify the stored seal;
- ULC-01 remains healthy as the writer and ULC-02 remains healthy-but-write-disabled for the final recovery snapshot.

The adapter runtime itself is no longer unavailable: both immutable workers are deployed and internally commissioned. Real Fabric execution remains **externally gated** until the other Fabric system supplies its Gateway/TLS/MSP/channel/chaincode/function handoff and a real transaction can be committed and queried. Do not substitute a mock ledger for that final acceptance. Adapter loss, OpenBao member loss, Fabric outage/backlog, and reconcile/drain behavior are proven later when a real Fabric target exists.

Next required checkpoint: **Phase 13B** in [13-backup-restore-and-disaster-recovery.md](#), then Phase 14 and Phase 14B.

Canonical source: [deployment/technology-guides/12-openbao.md](#). For any mutable production-status statement, check the dated baseline and verified live state.